

The Space Complexity of Learning-Unlearning Algorithms

author names withheld

Editor: Under Review for COLT 2025

Abstract

We study the memory complexity of machine unlearning algorithms that provide strong data deletion guarantees to the users. Formally, consider an algorithm for a particular learning *task* that initially receives a training dataset. Then, after learning, it receives data deletion requests from a subset of users (of arbitrary size), and the goal of unlearning is to perform the *task* as if the learner never received the data of deleted users. In this paper, we ask how many bits of storage are needed to be able to delete certain training samples, at a later time. We focus on the task of realizability testing, where the goal is to check whether the remaining training samples are realizable within a given hypothesis class $\mathcal{H}$.

Toward that end, we first provide a negative result showing that the VC dimension—a well-known combinatorial property of $\mathcal{H}$ that characterizes the amount of information needed for learning and representing the ERM hypothesis in the standard PAC learning task—is not a characterization of the space complexity of unlearning. In particular, we provide a hypothesis class with constant VC dimension (and Littlestone dimension), but for which, any unlearning algorithm for realizability testing needs to store $\Omega(n)$ -bits, where n denotes the size of the initial training dataset. In fact, we provide a stronger separation by showing that for any hypothesis class $\mathcal{H}$, the amount of information that the learner needs to store, so as to perform unlearning later, is lower bounded by the *eluder dimension* of $\mathcal{H}$, a combinatorial notion always larger than the VC dimension. We complement the lower bound with an upper bound, albeit in a stronger ticketed-memory model proposed by Ghazi et al. (2023). We show that for any class $\mathcal{H}$ with bounded eluder dimension, there exists a ticketed scheme that uses only $\tilde{O}(\text{Eluder}(\mathcal{H}))$ many bits of storage and these many sized tickets. Our work thus implies that the eluder dimension of a hypothesis class plays a fundamental role in the memory complexity of unlearning.

Lastly, we consider the setting where the number of deletions is *bounded* and show that in contrast to the unbounded setting, there exist unlearning schemes with sublinear (in n) storage for hypothesis classes with bounded *hollow star number*—a notion of complexity that is always smaller than the Eluder dimension.

Keywords: Unlearning, VC/ Littlestone dimension, space complexity, lower bounds

1. Introduction

A fundamental right that has emerged through recently proposed regulations such as European Union’s (GDPR), the California Consumer Privacy Act (CCPA), and Canada’s proposed Consumer Privacy Protection Act (CPPA), is the “right to be forgotten”. This principle allows individuals to demand removal of their personal data from a trained ML system. Though, this seems like a simple task at first glance, formalizing what it means to remove is challenging. In a typical large-scale ML system, the data is not only stored in its raw form in various training datasets, but can be memorized by the ML models in complicated ways, and can thus influence the behavior of the model towards other users, inadvertently leaking private information (Shokri et al., 2017; Carlini et al., 2022, 2023b,a; Ippolito et al., 2022).

Given the importance of this problem, considerable efforts have been dedicated in recent years to formalize what “forgetting” really means for interactions between individuals and data holders (Cohen and Nissim, 2020; Cohen et al., 2023; Garg et al., 2020; Cao and Yang, 2015). In the context of machine learning, this led to the notion of *machine unlearning*, first proposed by Cao and Yang (2015) where the goal is that, post-deletion, the ML model behaves as if the deleted data was never included in the ML model, to begin with. In order to circumvent the massive overhead of “retraining from scratch” every time a deletion is requested, there has been considerable research in more feasible strategies for machine unlearning (see Nguyen et al. (2022) and references therein).

While the training time aspect of machine unlearning has garnered significant attention in recent literature (Ginart et al., 2019; Wu et al., 2020; Golatkar et al., 2020a,b; Bourtole et al., 2021; Izzo et al., 2021; Neel et al., 2021; Sekhari et al., 2021; Jang et al., 2022; Huang and Canonne, 2023; Wang et al., 2023), both from the theoretical and practical perspective, our understanding of its memory complexity remains rudimentary. A few recent works have considered the memory footprint of machine unlearning, and the associated tradeoffs due to memory constraints (Sekhari et al., 2021; Ghazi et al., 2023). Ghazi et al. (2023) introduced the notion of central and ticketed learning-unlearning schemes, and mergeable hypothesis classes, and provided space-efficient unlearning schemes for various specific hypothesis classes. But there is a lack of a characterizations of memory complexity of unlearning. Toward bridging this gap, in this paper, we ask

Can natural combinatorial notions of the complexity of a class $\mathcal{H}$ be used to bound the information that needs to be retained for unlearning?

This question is natural, given the rich literature on characterizations for learnability in the standard machine learning setting (i.e. no data removal) by combinational notations e.g. VC dimension for PAC learning, Littlestone dimension for online learning, and the star number and eluder dimension for active learning.

In addition to theoretical interest, practically, in many modern ML settings, the vast size of training datasets often makes storage capacity a critical bottleneck. In this work, we focus on the memory complexity of machine unlearning for the task of realizability testing under deletions, where the goal is to just answer whether the remaining training data is consistent with the given hypothesis class or not. This simpler task serves as a useful theoretical primitive that captures various interesting aspects of the full-fledged unlearning problem—where the learner needs to return a hypothesis within a given hypothesis class—and the tools we develop for understanding space complexity for realizability testing are useful even when considering unlearning schemes for empirical risk minimization (See Appendix H for formal white-box reductions).

1.1. Our Contributions

We study the memory complexity of realizability testing under two models of unlearning: the central model and the ticketed model. In the central model, all the necessary information for unlearning is stored in a centralized memory (Definition 3). On the other hand, in the ticketed model, this necessary information is spread between a central memory and users corresponding to data points in the set (Definition 4). We summarize our contributions in Table 1 on page 3.

Sample Compression Schemes Do Not Suffice. Sample compression schemes (Littlestone and Warmuth, 1986; Floyd and Warmuth, 1995; Moran and Yehudayoff, 2016) are the standard notion of compression studied in learning theory. Here, one asks for a small subset of the dataset from

		Bounded VC Dim	Arbitrary # Deletions Bounded Littlestone Dim	Bounded Eluder Dim	k -Deletions
Central	Upper:	$O(n \log(\mathcal{Z}))$ Trivial; store the dataset	$O(n \log(\mathcal{Z}))$ Trivial; store the dataset	Open Problem 1	$O(\mathfrak{s}_o(\mathcal{H})^k k \log(\mathcal{Z}))$ (Theorem 24)
	Lower:	$\Omega(n)$ for 1D thresholds (Ghazi et al., 2023)	$\Omega(n)$ for some $\mathcal{H}$ (Theorem 7)	$\Omega(\epsilon(\mathcal{H}))$ (Theorem 10)	$\Omega(d^k)$ for d -dim linear seps. (Appendix F, Theorem 34)
Ticketed	Upper:	$O(n \log(\mathcal{Z}))$ Trivial; store the dataset	$O(n \log(\mathcal{Z}))$ Trivial; store the dataset	$O(\epsilon(\mathcal{H}) \log(\mathcal{Z}) \log(n))$ (Theorem 11) (Ghazi et al., 2023, $O(d \log(n))$ for d -bit mergeable $\mathcal{H}$)	Same as central
	Lower:	$\Omega(\epsilon n^{1-\epsilon})$ for some $\mathcal{H}$ with VC-dim $O(1/\epsilon)$ (Theorem 7)	$\Omega(\epsilon n^{1-\epsilon})$ for some $\mathcal{H}$ with LS-dim $O(1/\epsilon)$ (Theorem 7)	not possible to get $\Omega(\text{poly}(\epsilon(\mathcal{H})))$ (Theorem 15)	$\Omega(d^k)$ for d -dim linear seps. (Appendix F, Theorem 34)

Table 1: Summary of our main results for realizability testing, ignoring the dependence on ϵ and δ (Here, $\mathfrak{s}_o$ refers to the hollow star number and $\mathcal{Z}$ to the domain of the hypothesis class $\mathcal{H}$ (Definition 21). Note that for $\mathcal{H}$ denoting the set of all linear separators in d -dimensions, $\mathfrak{s}_o(\mathcal{H}) \leq d + 2$.)

which one can infer the labels of the entire dataset. This is intimately related to combinatorial dimensions such as the VC dimension and the Littlestone dimension. We show, in Section 3.2, that even the Littlestone dimension (and thus the smaller VC dimension) is not sufficient for small memory learning-unlearning schemes for realizability testing.

Version Space Compression. In Section 4.2, we define a new notion known as *version space compression*, which corresponds to an encoding from which one can recover the entire set of hypothesis consistent with a data set. This notion is motivated by the fact that keeping track of realizable hypothesis suffices (even in the central model) when there are only insertions to the data set. We extend this for deletions and show that version space compression schemes suffice for ticketed unlearning schemes (Lemma 17 and Theorem 13). Further, surprisingly, version space compression schemes are not sufficient for the central model (Theorem 10 and Section 19). This formalizes the intuition that for realizability testing, handling deletions from the data set is harder than handling additions to that dataset. In particular, we show any class with a C -bit version space compression has a $O(C \log n)$ -bit ticketed LU scheme on data sets of size n . Further, for the class of linear thresholds in one dimension, there is a version space compression storing 2 data points, while any central LU schemes needs linear (in n) memory.

Eluder Dimension and Version Space Compression. We explore version space compression further by connecting it to the *eluder dimension*, a notion of complexity arising from the study of sequential decision making. We show that classes with bounded eluder dimension have version space compression schemes of linear size in the eluder dimension (Theorem 18).

Along with the previously mentioned connection between version space compression and ticketed learning unlearning schemes, this establishes that classes with small eluder dimension have ticketed unlearning schemes. Furthermore, we complement this upper bound by providing a lower bound in the central model to any hypothesis class with unbounded eluder dimension. Particularly,

any central memory learning unlearning scheme for any class with eluder dimension ϵ requires $\Omega(\min\{n, \epsilon\})$ bits of memory (Theorem 10).

Bounded Deletions. Motivated by circumventing the strong space lower bounds in the central model, we study the setting where the number of deletions is bounded. Here, we provide a central memory learning-unlearning scheme with space complexity that grows with the *hollow star number* of $\mathcal{H}$ (Theorem 24). That is, for a class with hollow star number $\mathfrak{s}_o$, there is a central memory LU scheme that handles k deletions and requires $\tilde{O}(\mathfrak{s}_o^k)$ bits of memory. The hollow star number is always smaller than the eluder dimension and typically smaller than the Littlestone dimension. A class that witnesses this gap is the class of halfspaces in d dimension, which has hollow star number $O(d)$ but infinite eluder and Littlestone dimensions. In fact, for this natural class, we show that this theorem is essentially tight (Appendix F). For the class of halfspaces in d dimensions, any LU scheme (central or ticketed) that handles k deletions requires $\Omega(d^{k-1})$ bits of memory.

Connections to Empirical Risk Minimization. We further show that the tools we develop for designing space complexity lower/ upper bounds for LU schemes (and later, TiLU schemes) for realizability testing are useful even when considering unlearning schemes for empirical risk minimization. Appendix H is dedicated to understanding these connections in more detail and presents new upper and lower bounds in the context of unlearning schemes for ERM.

1.2. Technical Overview

Memory Complexity Lower Bounds. The space lower bounds for learning-unlearning schemes in the central-memory as well as the ticketed model follow a common recipe: construct a 1) collection of datasets indexed by a large secret/ string, and 2) set of unlearning requests such that outputs to these requests reveal the secret. We then argue that the auxiliary information stored for facilitating these unlearning requests can be used as a compression scheme for the secret; thus, the size of the secret becomes a lower bound for the memory needed for learning-unlearning schemes. The lower bounds for approximate-unlearning (see Definition 3) needs a bit more work as the secret is not revealed completely with the unlearning queries.

A key challenge arises while showing memory lower bounds in the ticketed model. Since we are aiming to bound the size of each ticket, an unlearning request with a large number of deletions inherently allows the algorithm more power (i.e. large effective memory). To show a lower bound through compression argument, we additionally need to ensure that all the deletion requests come from a small set of users/datapoints. In Theorem 7, we construct a hypotheses class with constant Littlestone dimension, such that any learning-unlearning scheme, even in the ticketed model, requires essentially the memory required to store the entire data set.

Version-Space Compression. One of the key contributions of this work is the introduction of the notion of a version space compression scheme for a hypothesis class $\mathcal{H}$. Given a dataset, a version space represents the set of all hypotheses from the $\mathcal{H}$ that are consistent with the dataset. A version space compression scheme is an algorithm that takes as input a data set S and outputs an encoding from which one can recover the version space corresponding to S . We show that version space compression schemes are essentially equivalent to a notion of compression known as mergeable compression (Ghazi et al., 2023), which corresponds to encoding of data sets from which one can infer whether the data set is realizable or not, with an additional property that encoding of the union of two data sets can be computed from the encoding of the two data sets. Using this equivalence,

we show that there is a ticketed unlearning scheme for any class with a version space compression scheme. We further use this perspective to expand upon the connection to eluder dimension by showing that the classes with bounded eluder dimension have bounded version space compression schemes. The compression of dataset S stores an eluding subsequence of S .

To complement this upper bound through version-space compression schemes, we also show that for classes that satisfy an additional structural assumption of having small Minimum Identification Set (MIS)—a small set such that knowing the labels of the points in the set uniquely identifies the hypothesis, one can convert a low memory exact-unlearning scheme in the central model to a version space compression scheme (Theorem 29). This result can be used to show that for the class of parities over $\{0, 1\}^d$ any unlearning scheme in the central model must require $\Omega(d^2)$ bits of memory even for data sets of size $O(d)$.

Bounded Deletions. All the lower bounds discussed above requires unlearning requests where the number of deletions grow polynomially with the size of the (original) training dataset. An interesting question remain as to whether the memory complexity of unlearning can be reduced when the number of deletions is bounded. We show that this is indeed the case; the memory complexity in the case of constant deletions is upper bounded by the hollow star number of the hypothesis class, even in the central memory model. The hollow star number is a combinatorial parameter that captures size of the largest unrealizable data set such that removing any one point from the data set makes it realizable.

Towards building an intuition for the results, we first note that for any unrealizable data set, there is a subset of size at most the hollow star number such that it is also unrealizable. This can be seen by greedily removing points from the data set until the removal of any point makes the data set realizable and the size of the set thus obtained is at most the hollow star number. Given this observation, note that the number of points whose removal from a data set makes it realizable is at most the hollow star number since the point must lie in the unrealizable set of size at most the hollow star number constructed above. This gives a simple central memory scheme that handles one deletion: store all the points whose removal makes the data set realizable. This idea can be applied recursively to show that the number of subsets of size k whose removal from a data set makes it realizable (we call such subset k -critical) is at most the hollow star number to the power of k .

We complement this upper bound by showing that for the class of linear separators in d dimension (whose hollow star number is $O(d)$), the memory complexity of unlearning for k deletions is lower bounded by $\Omega(d^k)$. In fact, we even show this lower bound in the ticketed model. The idea again is to construct a large family of data sets whose identity can be inferred using deletions on a small set of users/datapoints.

2. Formalization of Learning-Unlearning

Basic Notation. We will use $[m]$ to denote the set $\{1, 2, \dots, m\}$ for any $m \in \mathbb{N}$. For any set S and $p \leq |S| \in \mathbb{N}$, $\binom{S}{p} = \{T \subseteq S : |T| = p\}$ denotes the p -sized subsets of S . In this paper, we will focus on the setting of binary classification. Let $\mathcal{X}$ be a set of feature vectors. Unless stated otherwise, we study the case where $\mathcal{X}$ is finite. A hypothesis h over the domain $\mathcal{X}$ is a function $\mathcal{X} \rightarrow \{0, 1\}$. A collection of hypotheses is known as a hypothesis class, which we will denote by $\mathcal{H}$. A dataset $D \in (\mathcal{X} \times \{0, 1\})^*$ is a sequence of ordered pairs $\{(x_i, y_i)\}$, where $x_i \in \mathcal{X}$ and $y_i \in \{0, 1\}$. We will call y_i the label of x_i . For ease of notation, we will use $\mathcal{Z}$ to denote the set $(\mathcal{X} \times \{0, 1\})$. Let $\mathcal{W}$ be an arbitrary set. For a distribution $\mathbf{P} : \mathcal{W} \rightarrow [0, 1]$, we use $\mathbf{P}(\mathcal{W}')$ to denote the probability mass

over $\mathcal{W}' \subseteq \mathcal{W}$, that is, $\mathbf{P}(\mathcal{W}') = \sum_{w \in \mathcal{W}'} \mathbf{P}(w)$. We defer other standard definitions and notation to [Appendix B](#), and formalize the notion of a learning task and unlearning below.

Definition 1 (Learning task) *Let $\mathcal{W}$ be a set of possible outcomes. A learning task with respect to a hypothesis class $\mathcal{H}$ is denoted by function $f : \mathcal{Z}^* \rightarrow \mathcal{W}$ that takes in a dataset as input, and outputs a solution in range $\mathcal{W}$.*

We will focus on two learning tasks in this paper: realizability testing and Empirical Risk Minimization (ERM). For the realizability testing problem, the goal is to determine whether there exists a hypothesis that labels all the points in the dataset correctly.

Definition 2 (Realizability Testing) *Let $\mathcal{H}$ be a hypothesis class. A dataset $D = \{(x_i, y_i)\}_{i=1}^n$ is said to be $\mathcal{H}$ -realizable if there exists $h \in \mathcal{H}$ such that $y_i = h(x_i)$ for all $i \in [n]$.*

Realizability testing corresponds to the learning task f of deciding whether an input dataset $D = \{(x_i, y_i)\}_{i=1}^n$ is $\mathcal{H}$ -realizable or not. In particular, $\mathcal{W} = \{\text{Yes}, \text{No}\}$ and $f(D) = \text{Yes}$ if D is $\mathcal{H}$ -realizable and $f(D) = \text{No}$ otherwise.

We defer the definition of ERM to [Appendix H](#) since the majority of the main body of the paper focuses on realizability testing (many results trivially extend to the ERM setting; cf. [Appendix H](#) for details). Note that for the task of computing ERMs, the set $\mathcal{W}$ referred to in [Definition 1](#) denotes the set of Empirical Risk Minimizers within a class $\mathcal{H}$ for a given dataset S .

Definition 3 (Central-memory (ε, δ) -Learning-Unlearning (LU) scheme for a learning task) *Let $\varepsilon, \delta \in (0, 1)$. For a learning task $f : \mathcal{Z}^* \rightarrow 2^{\mathcal{W}}$ with respect to a hypothesis class $\mathcal{H}$, an (ε, δ) -LU scheme corresponds to a pair of randomized algorithms (Learn, Unlearn). For any dataset $D = \{(x_i, y_i)\}_{i=1}^n$ and index set $I \subseteq [n]$, the scheme guarantees*

- *Learn(D) outputs a solution in $f(D) \subseteq \mathcal{W}$, and some auxiliary information of the dataset $\text{aux}(D) \in \{0, 1\}^*$. Let $\mathbf{P}_D$ denote the distribution over solutions outputted by Learn(D).*
- *For any unlearning query denoted by a subset of dataset, $D_I = \{(x_i, y_i) : i \in I\}$, Unlearn(D_I, aux) outputs a solution in $\mathcal{W}$ such that, for all subsets $\mathcal{W}' \subseteq \mathcal{W}$,*

$$\Pr[\text{Unlearn}(D_I, \text{aux}(D)) \in \mathcal{W}'] \leq e^\varepsilon \mathbf{P}_{D \setminus D_I}(\mathcal{W}') + \delta.$$

Here, the randomness is over the auxiliary information outputted by the Learn algorithm and random bits used in the Unlearn algorithm.

The space complexity of an (ε, δ) -LU scheme is said to be $s(n)$ if for all n , on any input dataset D of size at most n , the learning-unlearning scheme satisfies $|\text{aux}| \leq s(n)$, that is, the number of bits of auxiliary information stored is at most $s(n)$.

Throughout the paper, when ε, δ are not important or are clear from the context, we will use the phrase LU scheme to refer to a central-memory (ε, δ) -LU scheme. Additionally, we will overload the notation LU scheme to indicate LU schemes for the learning task of realizability testing, which is the main focus of this paper. We call an LU scheme *non-trivial* if it uses space that scales sublinearly in the size of the input dataset. It is natural for $s(n)$ to have at least some mild dependence on the size of the dataset n since just keeping track of the number of points in the dataset requires

$\log(n)$ memory. n . Unless stated otherwise, we assume that $n \ll \min(|\mathcal{X}|, |\mathcal{H}|)$. We call a LU scheme to be *space-efficient* if its space complexity scales polynomially with $\log n$, $\log |\mathcal{H}|$ and $\log |\mathcal{X}|$. We discuss some simple schemes that we deem inefficient to justify the above parameters in [Appendix B.2](#).

In this paper, we explore schemes that use $\text{poly}(\log n, d(\mathcal{H}), \log |\mathcal{X}|)$ bits of storage, where $d(\mathcal{H})$ is some combinatorial measure of the hypothesis class $\mathcal{H}$. As it will turn out, sublinear (in n) memory LU schemes often do not exist, even for simple classes. Indeed, we will show in [Theorem 7](#) that there exist classes with constant VC and Littlestone dimensions, which do not admit sublinear-memory (ε, δ) -LU schemes for any $\delta < 1/2$. To this end, we also consider the ticketed model of unlearning, introduced in [Ghazi et al. \(2023\)](#), which generalizes central-memory LU schemes. Here, each datapoint is assumed to belong to a client, and the unlearning scheme is allowed to store additional metadata with each client.

Definition 4 (Ticketed (ε, δ) -Learning-Unlearning (TiLU) scheme) *For a learning task $f : \mathcal{Z}^* \rightarrow 2^{\mathcal{W}}$ w.r.t. a hypothesis class $\mathcal{H}$, an (ε, δ) -TiLU scheme corresponds to algorithms (Learn, Unlearn) such that for any dataset $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$ and index set $I \subseteq [n]$,*

- *Learn(D) returns a solution in $f(D) \subseteq \mathcal{W}$, and some auxiliary information of the dataset $(\text{aux}, \{\text{tx}_i\}_{i \in [n]})$. Here, $\text{aux} \in \{0, 1\}^*$ is stored in the central memory, while the ticket $\text{tx}_i \in \{0, 1\}^*$, associated with the sample (x_i, y_i) , is stored with the i -th user. Let $\mathbf{P}_D$ denote the distribution over solutions output by Learn(D).*
- *For any unlearning query given by a subset of the dataset, $D_I = \{(x_i, y_i)\} : i \in I$, using the auxiliary information in central memory and the tickets from users in I , Unlearn outputs a solution in $\mathcal{W}$ such that, for all $\mathcal{W}' \subseteq \mathcal{W}$, $\Pr[\text{Unlearn}(D_I, \text{aux}(D), \{\text{tx}_i\}_{i \in I}) \in \mathcal{W}'] \leq e^\varepsilon \mathbf{P}_{D \setminus D_I}(\mathcal{W}') + \delta$.*

Here, the randomness is over the random bits of Learn and Unlearn algorithms. The space complexity of an (ε, δ) -TiLU scheme is said to be $s(n)$ if on any input dataset D of size at most n , the learning-unlearning scheme satisfies $\max\{|\text{aux}|, \max_{i \in [n]} |\text{tx}_i|\} \leq s(n)$, i.e., the ticket size and number of auxiliary bits at the server are at most $s(n)$.

The tickets are additional information the learner provides to each user, in order to reduce the memory requirement at the server. In particular, to unlearn some set of points indexed by $I \subseteq [n]$, the learner can aggregate tickets from all the users in I to carry out unlearning. The motivation for this model is that the learner has access to more information to unlearn larger subsets of the dataset.

3. Does Bounded VC Dimension Imply Unlearning?

It is natural to ask whether classes $\mathcal{H}$ with bounded VC dimension (or Littlestone dimension) support sublinear-memory unlearning schemes for $\mathcal{H}$ -realizability testing. These complexity measures are known to be related to sample compression schemes for learning ([Floyd and Warmuth, 1995](#); [Moran and Yehudayoff, 2016](#)). We defer the formal definition of these dimensions to [Appendix B.1](#).

3.1. Unlearning with Sublinear Memory Implies Bounded VC Dimension

We first show that any class, for which there exists a central-model learning-unlearning algorithm with non-trivial space complexity, must have bounded VC dimension, that is, it must also be learn-

able under PAC learning. This results follows as a corollary of the following theorem, which shows that any bounded VC class needs a certain number of bits in the central memory for any LU scheme.

Theorem 5 *Let $\varepsilon \in [0, 1]$ and $\delta \in [0, 1/2)$ be constants. For all hypothesis classes $\mathcal{H}$, any (ε, δ) -LU scheme for $\mathcal{H}$ -realizability testing must use $\text{VC}(\mathcal{H}) \cdot (1 - H(\delta))$ bits of storage, even for datasets of size $n = O(\text{VC}(\mathcal{H}))$.*

Corollary 6 (LU scheme $\implies$ Bounded VC dimension) *Let $\varepsilon \in [0, 1]$ and $\delta \in [0, 1/2)$ be constants. Consider any hypothesis class $\mathcal{H}$ for which there exists a $C \in \mathbb{N}$ and an (ε, δ) -LU scheme with space complexity of at most $s(n) = C \log(n)$ bits on datasets of size at most n . Then, the VC dimension of $\mathcal{H}$ is at most $O_\delta(C \log(C))$.*

The above corollary is the contrapositive of [Theorem 5](#); indeed, if the VC dimension of $\mathcal{H}$ were some $d \gg C$, then by [Theorem 5](#), there must exist a dataset of size $2d$ such that any LU scheme requires a space complexity of $\Omega(d)$ bits. On this dataset, the LU scheme in [Corollary 6](#) has a space complexity upper bounded by $m = C \log(2d)$. We get a contradiction for $d = O(C \log(C))$. This result can be extended to the case where the space complexity of the schemes scales as sublinearly with n ; for a hypothesis class $\mathcal{H}$, if there exists a LU scheme with space complexity at most $Cf(n)$, where $f = (\cdot)^\alpha$ for a constant $\alpha \in (0, 1)$, then $\mathcal{H}$ has VC dimension at most $O(C^{1/(1-\alpha)})$. Due to space constraints, all the formal proofs are deferred to the Appendix.

3.2. Finite VC Dimension Does Not Imply Unlearning with Sublinear Memory

[Corollary 6](#) shows that the existence of a sublinear LU scheme implies that the hypothesis class must have finite VC dimension. In this section, we show that the converse is not true. The main result of this section is the following surprising lower bound: there exist hypothesis classes that have constant Littlestone dimension (and thereby constant VC dimension), but any LU scheme that supports unlearning a constant number of points requires a space complexity of $\Omega(n)$ bits. Moreover, these results extends to a near-linear space complexity lower bound for TiLU schemes.

Theorem 7 *For any $\beta \in (0, 1)$, there exists a hypothesis class $\mathcal{H}$ with Littlestone dimension (and thereby, VC dimension) at most $1/\beta + 1$, such that*

- *For any (ε, δ) -LU scheme for $\mathcal{H}$ -realizability testing, the space complexity is at least $\Omega((1 - H(\delta)) \cdot n)$ bits,*
- *For any (ε, δ) -TiLU scheme for $\mathcal{H}$ -realizability testing, the space complexity is at least $\Omega(\beta(1 - H(\delta)) \cdot n^{1-\beta})$ bits.*

Thus, the VC dimension (or Littlestone dimension) does not characterize whether there exists a LU or TiLU scheme with space complexity scaling as $\text{poly}(\log(n), \log(|\mathcal{X}|))$. In the next section, we tune into a property of every hypothesis class that generalizes this lower bound.

4. Toward a Characterization of the Space Complexity of Unlearning via Eluder Dimension

The key idea behind the lower bound in [Theorem 7](#) is to show that it is possible to find a sequence of points such that under an appropriate unlearning query, the presence or absence of the i^{th} point

in the sequence decides whether the dataset is separable or not. In particular, consider the sequence of points $\{(x_i, 0) : i \in [m]\}$ in the dataset. For any $i \in [m]$, consider $S = \{(x_j, 0) : j \leq i\}$; there exist hypotheses $h, h' \in \mathcal{H}$ which agree with all the labels in S , but don't agree on their labels of the next point, i.e., $0 = h(x_{i+1}) \neq h'(x_{i+1}) = 1$. By choosing the unlearning query carefully, we force h' to be the only function that potentially agrees with the remaining data, i.e. $\{(x_i, 0) : i \in [m]\}$ and thereby, realizability testing can be used to assess whether $(x_{i+1}, 0)$ exists in the dataset or not.

Such sequences have been considered in learning theory under the name of “eluder sequences” (Russo and Van Roy, 2013; Hanneke, 2024). The presence of such a long sequences is a problem for an unlearning scheme - knowing which of $(x_1, 0), \dots, (x_i, 0)$ exist in the dataset provides no indication as to whether $(x_{i+1}, 0)$ exists in the dataset or not, and the unlearning scheme ends up having to store information about all such points to be able to answer the unlearning queries.

We show that the existence of a long eluder sequence for a hypothesis class implies that any LU scheme for the class has high space complexity on worst-case datasets. To this end, we define the eluder dimension of a class (Russo and Van Roy, 2013; Hanneke, 2024). First, we recall the definition of the version space.

Definition 8 (Version Space) *Let $\mathcal{H}$ be a hypothesis class over a domain $\mathcal{X}$. Let D be a data set of labeled points. The version space of D with respect $\mathcal{H}$, denoted by $\mathcal{H}(D)$, is defined as the set of hypotheses consistent with these datapoints i.e. $\mathcal{H}(D) = \{h \in \mathcal{H} \mid h(x) = y, \forall (x, y) \in D\}$.*

Definition 9 (Eluder Dimension) *Let $\mathcal{H}$ be a hypothesis class over a domain $\mathcal{X}$. A sequence of labeled points $(x_1, y_1), \dots, (x_\ell, y_\ell)$ is said to be an eluder sequence if for each $i \leq \ell - 1$, the version space of $\{(x_1, y_1), \dots, (x_i, y_i)\}$ contains hypotheses h, h' such that $h(x_{i+1}) \neq h'(x_{i+1})$. The eluder dimension of $\mathcal{H}$ is defined as the largest value of ℓ with an eluder sequence of length ℓ .*

Theorem 10 *Let $\varepsilon \in [0, 1]$ and $\delta \in [0, 1/2]$ be constants. For any hypothesis class $\mathcal{H}$ with eluder dimension $\epsilon(\mathcal{H})$, the space complexity of any (ε, δ) -LU scheme on datasets of size n , is at least $\Omega(\min(n, \epsilon(\mathcal{H})))$ (where Ω hides a constant depending on δ).*

4.1. Ticketed Learning-Unlearning Schemes Under Bounded Eluder Dimension

In this section, we show that for classes with bounded eluder dimension, there is a space-efficient ticketed scheme with space complexity¹ $O(\epsilon(\mathcal{H}) \log n \log |\mathcal{X}|)$.

Theorem 11 (TiLU Scheme under bounded eluder dimension) *For any hypothesis class $\mathcal{H}$ with eluder dimension $\epsilon(\mathcal{H})$, there exists a $(0, 0)$ -TiLU scheme with ticket size $O(\epsilon(\mathcal{H}) \log(|\mathcal{Z}|) \log(n))$ and central memory size of 1 bit.*

Note that we provide deterministic learning-unlearning schemes in Theorem 11, and thus $\epsilon = 0$ and $\delta = 0$, following Definition 4. The proof of the above theorem goes through compression schemes, which we define over this and next subsection. The following definition is a straightforward extension of mergeable hypothesis classes in Ghazi et al. (2023), but for realizability testing.

Definition 12 (Mergeable hypothesis class for realizability testing) *A Hypothesis class $\mathcal{H} \subseteq \{\mathcal{X} \mapsto \mathcal{Y}\}$ is said to be C -bit mergeable for realizability testing if there exist methods `Encode`, `Merge` and `Decode` such that*

1. The space lower bound of Theorem 10 can be strengthened to $\epsilon(\mathcal{H}) \log(n/\epsilon(\mathcal{H}))$ bits for $(0, 0)$ -LU schemes. We give the full proof in Appendix.

- **Encode** : $(\mathcal{X} \times \mathcal{Y})^* \mapsto \{0, 1\}^C$ is a permutation-invariant encoding of its input into C bits.
- **Decode** : $\{0, 1\}^C \mapsto \{\text{Yes}, \text{No}\}$ such that $\text{Decode}(\text{Encode}(S)) = \text{Yes}$ if S is $\mathcal{H}$ -realizable, and is **No** otherwise.
- **Merge** : $\{0, 1\}^C \times \{0, 1\}^C \mapsto \{0, 1\}^C$ such that for all datasets $S_1, S_2 \in (\mathcal{X} \times \mathcal{Y})^*$, it holds that $\text{Merge}(\text{Encode}(S_1), \text{Encode}(S_2)) = \text{Encode}(S_1 \cup S_2)$.

The following theorem show that a mergeable class leads to a ticketed LU scheme. The proof of [Theorem 13](#) is given in [Appendix D.6](#) for completeness.

Theorem 13 *Suppose $\mathcal{H}$ is a C -bit mergeable hypothesis class, then it admits a $(0, 0)$ -TiLU realizability testing scheme, with both the ticket size and auxiliary bits bounded by $O(C \log n)$.*

A key question is when mergeable compressions exist. The next lemma addresses this in terms of the eluder dimension. The proof of the above lemma is deferred to [Section 4.2](#). It goes through version-space compression, which we argue to be an essential as well as sufficient notion of compression for exact unlearning under the central and ticketed model respectively. Precisely, the above lemma follows from [Lemma 17](#) and [Theorem 18](#).

Lemma 14 *Any class $\mathcal{H}$ with eluder dimension $\epsilon(\mathcal{H})$ is $O(\epsilon(\mathcal{H}) \log(|\mathcal{Z}|))$ -bit mergeable.*

Can we hope to get a general lower bound for ticketed model? [Theorem 10](#) shows that any central LU algorithm needs $\Omega(\epsilon(\mathcal{H}) \log n)$ memory. On the other hand, in [Theorem 11](#), we showed an unlearning scheme with tickets of size $\tilde{O}(\epsilon(\mathcal{H}) \log n)$. A natural question is whether the eluder dimension characterizes unlearning in the central or the ticketed unlearning model. We show that such a characterization impossible for the latter, while leaving the question open for the former.

Theorem 15 *There exists a hypothesis class $\mathcal{H}$ with VC dimension d (hence, eluder dimension at least d), admitting a TiLU scheme with space complexity at most $O(\max\{\log(d), \log(n), \log(|\mathcal{X}|)\})$.*

Open Problem 1 *Let $\mathcal{H}$ be a hypothesis class with eluder dimension $\epsilon(\mathcal{H})$. Under the central memory model, does there exist a learning-unlearning scheme for $\mathcal{H}$ with space complexity $\text{poly}(\epsilon(\mathcal{H}), \log(|\mathcal{X}|), \log(n))$?*

4.2. A Compression Perspective

As discussed in [Section 3](#), both VC and Littlestone dimensions aren't sufficient to justify the hardness of memory-bounded unlearning. Therefore, sample compression schemes, well studied in the literature ([Floyd and Warmuth, 1995](#); [Moran and Yehudayoff, 2016](#)), aren't sufficient for unlearning in the central as well as ticketed model. In this section, we introduce the concept of version-space compression and provide evidence for why this is the right notion of compression for the problem of exact learning-unlearning.

Definition 16 (Version-Space compression) *Let $\mathcal{H}$ be a hypothesis class over domain $\mathcal{X}$ and range $\mathcal{Y}$. A C -bit version-space compression scheme for $\mathcal{H}$, represented by a pair of functions (Enc, Dec) , is defined as follows:*

- Given a set of samples $\mathcal{S} = \{(x_1, y_1), (x_1, y_2), \dots, (x_r, y_r)\}$ as input, the encoding function $\text{Enc} : (\mathcal{X} \times \mathcal{Y})^* \rightarrow \{0, 1\}^C$ outputs a C -bit compression $\text{Enc}(\mathcal{S})$.
- On a C -bit string, the decoding function $\text{Dec} : \{0, 1\}^C \rightarrow 2^{\mathcal{H}}$ outputs a subset of the hypothesis $\mathcal{H}$, such that for all sample sets $\mathcal{S}$, $\text{Dec}(\text{Enc}(\mathcal{S})) = \mathcal{H}(\mathcal{S})$, where $\mathcal{H}(\mathcal{S}) = \{h \in \mathcal{H} \mid h(x) = y \text{ for all } (x, y) \in \mathcal{S}\}$.

First, we show that any version-space compression scheme implies a mergeable compression scheme and vice-versa. Combined with [Theorem 13](#), we will show that if a hypothesis class $\mathcal{H}$ admits a C -bit version-space compression scheme, then there exists a TiLU algorithm with ticket size at most $O(C \log n)$.

Lemma 17 (Version-Space Compression $\implies$ Mergeable Compression) *Suppose a hypothesis class $\mathcal{H}$ has a C -bit version-space compression scheme, given by the functions Enc_{vs} and Dec_{vs} , then $\mathcal{H}$ is a C -bit mergeable hypothesis class for realizability testing.*

Next, we show that any hypothesis class over a finite domain $\mathcal{X}$, having finite eluder dimension admits a finite version-space compression scheme.

Theorem 18 *Any hypothesis class $\mathcal{H}$ has a $O(\epsilon(\mathcal{H}) \log(|\mathcal{Z}|))$ -bit version-space compression scheme, where $\epsilon(\mathcal{H})$ denotes the eluder dimension of $\mathcal{H}$.*

Remark 19 *Note that there exists $\mathcal{H}$ with unbounded eluder dimension, but admitting version-space compression schemes that store constant number of samples (for e.g. 1d-thresholds).*

Next, we show that version-space compression is necessary for ticketed scheme upper bounds that go through [Theorem 13](#).

Lemma 20 (Mergeable Compression $\implies$ Version-Space Compression) *If a class $\mathcal{H}$ is C -bit mergeable for realizability testing, then it also admits a C -bit version-space compression scheme.*

[Lemma 17](#) along with [Theorem 13](#) implies that a version-space compression scheme for a hypothesis class $\mathcal{H}$ can be used to construct a TiLU algorithm for $\mathcal{H}$. While [Theorem 15](#) was used to show that eluder dimension fails to completely characterize the memory complexity of TiLU schemes, it also shows that a version-space compression scheme is not necessary for ticketed unlearning. On the other hand, we show in [Theorem 29](#) in [Appendix D.8](#) that version-space compression is indeed necessary for unlearning in the central model.

5. Unlearning Small Deletion Sets

In this section, we consider the setting where only a bounded number of points are included in a deletion request, and study the space complexity of an LU scheme for realizability testing, as a function of the number of points. Formally, a request provides a list of at most k -points to be deleted and the goal of `Unlearn` function is to determine whether the remaining dataset is realizable.

From [Theorem 7](#), we observe that even in the case of bounded deletions, the VC-dimension of a hypothesis class fails to characterize whether there exist LU schemes with sublinear space complexity. Here, we adopt an alternative notion of complexity – hollow star number, and show that for function classes with hollow star number $\mathfrak{s}_\circ$, there exists an unlearning scheme with space complexity $O(\mathfrak{s}_\circ^k \log(|\mathcal{Z}|))$. A formal definition of the hollow star number is provided below:

Definition 21 (Hollow star number; Hanneke and Yang (2015)) *The hollow-star number of a class $\mathcal{H}$ over a domain $\mathcal{X}$, denoted by $\mathfrak{s}_o(\mathcal{H})$, is defined as the largest $\ell \in \mathbb{N}$ for which there exists a set of points $\mathcal{S} = \{(x_1, y_1), \dots, (x_\ell, y_\ell)\}$ such that $\mathcal{S}$ is not realizable via $\mathcal{H}$, but for every $i \in \ell$, the one step neighbor of $\mathcal{S}$ is realizable via $\mathcal{H}$, or equivalently, there exists a hypothesis h_i such that $h_i(x_i) = \bar{y}_i$ and $h_i(x_j) = y_j$ for all $j \neq i$, where $\bar{y}_i$ denotes the complement of the label y_i .*

When clear from the context, we use $\mathfrak{s}_o$ to denote the hollow star number of the class $\mathcal{H}$. Notably, the hollow star number is always upper bounded by the star number (Lemma 41), and for hypothesis classes such as linear separators, the hollow star number is bounded by its dimension (Lemma 31), while the star number is unbounded. Next, we show that for any unrealizable set, there exists an unrealizable subset of size bounded by the hollow star number.

Lemma 22 *Let $S = \{(x_1, y_1), \dots, (x_\ell, y_\ell)\}$ be unrealizable. Then, there exists a subset $S' = \{(x'_1, y'_1), \dots, (x'_m, y'_m)\} \subset S$ which is also unrealizable and furthermore, $|S'| \leq \mathfrak{s}_o$.*

Next, we define a class of subsets of an unrealizable set which form a core part of our unlearning procedure. These will correspond to subsets of points whose removal from the dataset allows for realizable classification of the dataset. Furthermore, they satisfy the additional desirable property that any subset whose removal renders the dataset realizable *must* contain one of these critical sets. Hence, it suffices to simply store the set of k -critical sets. The main contribution of this section is showing that this quantity may be bounded in terms of the hollow star number.

Definition 23 (k -critical sets) *Let $\mathcal{X}, \mathcal{H}$ denote a domain space and a hypothesis class respectively. For a set $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$ that is unrealizable with respect to $\mathcal{H}$, we say that a size- k subset, $Q \subset S$ ($|Q| = k$), is k -critical if $S \setminus Q$ is realizable and furthermore, for any proper subset $Q' \subset Q$ ($|Q'| < |Q|$), $S \setminus Q'$ is unrealizable.*

In the proof of the following theorem, we provide a bound on the number of l -critical sets in terms of the hollow star number which implies the existence of a small-sized unlearning scheme.

Theorem 24 *Let $\mathcal{H}$ be a hypothesis class over a domain $\mathcal{X}$ and $k > 0$. Then, there exists a $(0, 0)$ -LU-scheme for realizability testing, when deletion queries contain at most k -points, with space complexity at most $O(\mathfrak{s}_o^k \cdot k \log(|\mathcal{Z}|))$.*

This upper bound is nontrivial because there are classes for which the Eluder dimension is unbounded, but the star number itself is bounded. We show that this space complexity is tight for the special class of linear separators (Appendix F).

Open Problem 2 *Suppose $\mathcal{H}$ is a hypothesis class with hollow star number $\mathfrak{s}_o$. Does there exist a TiLU scheme for $\mathcal{H}$, for arbitrary number of deletions, where both the ticket size and auxiliary memory size is bounded by $\text{poly}(\mathfrak{s}_o^{\mathfrak{s}_o}, \log(|\mathcal{Z}|), \log(|n|))$?*

Conclusion. In this paper, we studied the memory complexity of testing the realizability of a data set with respect to a hypothesis class under unlearning constraints. We investigated the relationship between various complexity measures of the hypothesis class and the memory required for learning-unlearning schemes. One interesting direction for future research is to investigate whether low-memory LU schemes exist for learning tasks beyond realizability testing and empirical risk minimization, especially when the learner is allowed to output a hypothesis within constant error of the ERM; for bounded deletions, algorithms from privacy literature may be useful for this setting.

References

- Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 20–29, 1996.
- Hagit Attiya, Michael A Bender, Martin Farach-Colton, Rotem Oshman, and Noa Schiller. History-independent concurrent objects. *arXiv preprint arXiv:2403.14445*, 2024.
- Paul Beame, Shayan Oveis Gharan, and Xin Yang. Time-space tradeoffs for learning finite functions from random evaluations, with applications to polynomials. In *Conference On Learning Theory*, pages 843–856. PMLR, 2018.
- Nora Belrose, David Schneider-Joseph, Shauli Ravfogel, Ryan Cotterell, Edward Raff, and Stella Biderman. Leace: Perfect linear concept erasure in closed form. *Advances in Neural Information Processing Systems*, 36, 2023.
- Michael A Bender, Martín Farach-Colton, Michael T Goodrich, and Hanna Komlós. History-independent dynamic partitioning: Operation-order privacy in ordered data structures. *Proceedings of the ACM on Management of Data*, 2(2):1–27, 2024.
- Moise Blanchard. Gradient descent is pareto-optimal in the oracle complexity and memory tradeoff for feasibility problems, 2024. URL <https://arxiv.org/abs/2404.06720>.
- Guy E Blelloch and Daniel Golovin. Strongly history-independent hashing with applications. In *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS’07)*, pages 272–282. IEEE, 2007.
- Lucas Bourtole, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. In *proceedings of the 42nd IEEE Symposium on Security and Privacy*, SP ’21. IEEE Computer Society, 2021.
- Olivier Bousquet, Steve Hanneke, Shay Moran, and Nikita Zhivotovskiy. Proper learning, helly number, and an optimal SVM bound. In Jacob D. Abernethy and Shivani Agarwal, editors, *Conference on Learning Theory, COLT 2020, 9-12 July 2020, Virtual Event [Graz, Austria]*, volume 125 of *Proceedings of Machine Learning Research*, pages 582–609. PMLR, 2020. URL <http://proceedings.mlr.press/v125/bousquet20a.html>.
- Mark Braverman, Gillat Kol, Shay Moran, and Raghuvansh R. Saxena. Convex set disjointness, distributed learning of halfspaces, and LP feasibility. *CoRR*, abs/1909.03547, 2019. URL <http://arxiv.org/abs/1909.03547>.
- Jonathan Brophy and Daniel Lowd. Machine unlearning for random forests. In *Proceedings of the 38th International Conference on Machine Learning, ICML ’21*, pages 1092–1104. JMLR, Inc., 2021.
- Niv Buchbinder and Erez Petrank. Lower and upper bounds on obtaining history independence. In *Advances in Cryptology-CRYPTO 2003: 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003. Proceedings 23*, pages 445–462. Springer, 2003.

- Yinzhi Cao and Junfeng Yang. Towards making systems forget with machine unlearning. In *2015 IEEE symposium on security and privacy*, pages 463–480. IEEE, 2015.
- Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramèr. Membership inference attacks from first principles. In *S & P*, pages 1897–1914, 2022.
- Nicholas Carlini, Jamie Hayes, Milad Nasr, Matthew Jagielski, Vikash Sehwal, Florian Tramèr, Borja Balle, Daphne Ippolito, and Eric Wallace. Extracting training data from diffusion models. *arXiv preprint arXiv:2301.13188*, 2023a.
- Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramèr, and Chiyuan Zhang. Quantifying memorization across neural language models. In *ICLR*, 2023b.
- Gert Cauwenberghs and Tomaso Poggio. Incremental and decremental support vector machine learning. *NIPS*, 2000.
- CCPA. California consumer privacy act (ccpa). <https://oag.ca.gov/privacy/ccpa>.
- Eli Chien, Haoyu Wang, Ziang Chen, and Pan Li. Langevin unlearning: A new perspective of noisy gradient descent for machine unlearning. *arXiv preprint arXiv:2401.10371*, 2024.
- Rishav Chourasia, Neil Shah, and Reza Shokri. Forget unlearning: Towards true data-deletion in machine learning. In *ICML*, 2023.
- Aloni Cohen and Kobbi Nissim. Towards formalizing the gdpr notion of singling out. *Proceedings of the National Academy of Sciences*, 117(15):8344–8352, 2020.
- Aloni Cohen, Adam Smith, Marika Swanberg, and Prashant Nalini Vasudevan. Control, confidentiality, and the right to be forgotten. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 3358–3372, 2023.
- R Dennis Cook and Sanford Weisberg. Characterizations of an empirical influence function for detecting influential cases in regression. *Technometrics*, 22(4):495–508, 1980.
- T Cover and M Hellman. The two-armed-bandit problem with time-invariant finite memory. *IEEE Transactions on Information Theory*, 16(2):185–195, 1970.
- Thomas M Cover. Hypothesis testing with finite statistics. *The Annals of Mathematical Statistics*, 40(3):828–835, 1969.
- Thomas M Cover, Michael A Freedman, and Martin E Hellman. Optimal finite memory learning algorithms for the finite sample problem. *Information and Control*, 30(1):49–85, 1976.
- Min Du, Zhi Chen, Chang Liu, Rajvardhan Oak, and Dawn Song. Lifelong anomaly detection through unlearning. In *CCS*, pages 1283–1297, 2019.
- Yonatan Dukler, Benjamin Bowman, Alessandro Achille, Aditya Golatkar, Ashwin Swaminathan, and Stefano Soatto. SAFE: Machine unlearning with shard graphs. *arXiv preprint arXiv:2304.13169*, 2023.

- Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Proceedings of the 3rd Conference on Theory of Cryptography*, TCC '06, pages 265–284, Berlin, Heidelberg, 2006. Springer.
- Thorsten Eisenhofer, Doreen Riepel, Varun Chandrasekaran, Esha Ghosh, Olga Ohrimenko, and Nicolas Papernot. Verifiable and provably secure machine unlearning. *arXiv preprint arXiv:2210.09126*, 2022.
- Ronen Eldan and Mark Russinovich. Who’s harry potter? approximate unlearning in llms. *arXiv preprint arXiv:2310.02238*, 2023.
- Philippe Flajolet and G Nigel Martin. Probabilistic counting algorithms for data base applications. *Journal of computer and system sciences*, 31(2):182–209, 1985.
- Sally Floyd and Manfred Warmuth. Sample compression, learnability, and the vapnik-chervonenkis dimension. *Machine learning*, 21(3):269–304, 1995.
- Sanjam Garg, Shafi Goldwasser, and Prashant Nalini Vasudevan. Formalizing data deletion in the context of the right to be forgotten. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 373–402. Springer, 2020.
- Sumegha Garg, Ran Raz, and Avishay Tal. Extractor-based time-space lower bounds for learning. In *Proceedings of the 50th Annual ACM Symposium on the Theory of Computing*, STOC '18, pages 990–1002, New York, NY, USA, 2018. ACM.
- GDPR. Regulation (EU) 2016/679 of the European parliament and of the council of 27 April 2016. *Official Journal of the European Union*.
- Badih Ghazi, Pritish Kamath, Ravi Kumar, Pasin Manurangsi, Ayush Sekhari, and Chiyuan Zhang. Ticketed learning-unlearning schemes, 2023.
- Antonio Ginart, Melody Guan, Gregory Valiant, and James Y Zou. Making AI forget you: Data deletion in machine learning. In *Advances in Neural Information Processing Systems 32*, NeurIPS '19, pages 3518–3531. Curran Associates, Inc., 2019.
- Jonathan Godin and Philippe Lamontagne. Deletion-compliance in the absence of privacy. In *2021 18th International Conference on Privacy, Security and Trust (PST)*, pages 1–10. IEEE, 2021.
- Shashwat Goel, Ameya Prabhu, Amartya Sanyal, Ser-Nam Lim, Philip Torr, and Ponnurangam Kumaraguru. Towards adversarial evaluations for inexact machine unlearning. *arXiv preprint arXiv:2201.06640*, 2022.
- Shashwat Goel, Ameya Prabhu, Philip Torr, Ponnurangam Kumaraguru, and Amartya Sanyal. Corrective machine unlearning. *arXiv preprint arXiv:2402.14015*, 2024.
- Aditya Golatkar, Alessandro Achille, and Stefano Soatto. Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020a.

- Aditya Golatkar, Alessandro Achille, and Stefano Soatto. Forgetting outside the box: Scrubbing deep networks of information accessible from input-output observations. *arXiv:2003.02960*, 2020b.
- Aditya Golatkar, Alessandro Achille, Avinash Ravichandran, Marzia Polito, and Stefano Soatto. Mixed-privacy forgetting in deep networks. In *Proceedings of the 2021 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, CVPR '21, pages 792–801. IEEE Computer Society, 2021.
- Laura Graves, Vineel Nagisetty, and Vijay Ganesh. Amnesiac machine learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 11516–11524, 2021.
- Chuan Guo, Tom Goldstein, Awni Hannun, and Laurens Van Der Maaten. Certified data removal from machine learning models. In *Proceedings of the 37th International Conference on Machine Learning*, ICML '20, pages 3832–3842. JMLR, Inc., 2020.
- Varun Gupta, Christopher Jung, Seth Neel, Aaron Roth, Saeed Sharifi-Malvajerdi, and Chris Waites. Adaptive machine unlearning. In *Advances in Neural Information Processing Systems 34*, NeurIPS '21, pages 16319–16330. Curran Associates, Inc., 2021.
- Steve Hanneke. *Theoretical foundations of active learning*. Carnegie Mellon University, 2009.
- Steve Hanneke. The star number and eluder dimension: Elementary observations about the dimensions of disagreement. *J. Mach. Learn. Res.*, 247, 2024.
- Steve Hanneke and Liu Yang. Minimax analysis of active learning. *J. Mach. Learn. Res.*, 16(1): 3487–3602, 2015.
- Jason D Hartline, Edwin S Hong, Alexander E Mohr, William R Pentney, and Emily C Rocke. Characterizing history independent data structures. *Algorithmica*, 42:57–74, 2005.
- Martin Edward Hellman. *Learning with finite memory*. Stanford University, 1969.
- Yiyang Huang and Clément L Canonne. Tight bounds for machine unlearning via differential privacy. *arXiv:2309.00886*, 2023.
- Daphne Ippolito, Florian Tramèr, Milad Nasr, Chiyuan Zhang, Matthew Jagielski, Katherine Lee, Christopher A Choquette-Choo, and Nicholas Carlini. Preventing verbatim memorization in language models gives a false sense of privacy. *arXiv preprint arXiv:2210.17546*, 2022.
- Zachary Izzo, Mary Anne Smart, Kamalika Chaudhuri, and James Zou. Approximate data deletion from machine learning models. In *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2021.
- Joel Jang, Dongkeun Yoon, Sohee Yang, Sungmin Cha, Moontae Lee, Lajanugen Logeswaran, and Minjoon Seo. Knowledge unlearning for mitigating privacy risks in language models. *arXiv preprint arXiv:2210.01504*, 2022.
- Victor Kac and Pokman Cheung. *q-Binomial Coefficients and Linear Algebra over Finite Fields*, pages 21–26. Springer New York, New York, NY, 2002. ISBN 978-1-4613-0071-7. doi: 10.1007/978-1-4613-0071-7_7. URL https://doi.org/10.1007/978-1-4613-0071-7_7.

- Masayuki Karasuyama and Ichiro Takeuchi. Multiple incremental decremental learning of support vector machines. *IEEE Transactions on Neural Networks*, 21(7):1048–1059, 2010.
- Gillat Kol, Ran Raz, and Avishay Tal. Time-space hardness of learning sparse parities. In *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*, pages 1067–1080, 2017.
- Satyapriya Krishna, Jiaqi Ma, and Himabindu Lakkaraju. Towards bridging the gaps between the right to explanation and the right to be forgotten. In *ICML*, 2023.
- Meghdad Kurmanji, Peter Triantafillou, and Eleni Triantafillou. Towards unbounded machine unlearning. *arXiv preprint arXiv:2302.09880*, 2023.
- Gene Li, Prithish Kamath, Dylan J Foster, and Nati Srebro. Understanding the eluder dimension. *Advances in Neural Information Processing Systems*, 35:23737–23750, 2022.
- Nick Littlestone. Learning quickly when irrelevant attributes abound: A new linear-threshold algorithm. *Machine learning*, 2:285–318, 1988.
- Nick Littlestone and Manfred Warmuth. Relating data compression and learnability. *Unpublished manuscript*, 1986.
- Annie Marsden, Vatsal Sharan, Aaron Sidford, and Gregory Valiant. Efficient convex optimization requires superlinear memory. In *Conference on Learning Theory*, pages 2390–2430. PMLR, 2022.
- Andrew McGregor. Graph stream algorithms: a survey. *ACM SIGMOD Record*, 43(1):9–20, 2014.
- Jayadev Misra and David Gries. Finding repeated elements. *Science of computer programming*, 2(2):143–152, 1982.
- Shay Moran and Amir Yehudayoff. Sample compression schemes for vc classes. *Journal of the ACM (JACM)*, 63(3):1–10, 2016.
- Dana Moshkovitz and Michal Moshkovitz. Mixing implies lower bounds for space bounded learning. In *Conference on Learning Theory*, pages 1516–1566. PMLR, 2017.
- Dana Moshkovitz and Michal Moshkovitz. Entropy samplers and strong generic lower bounds for space bounded learning. In *Proceedings of the 9th Conference on Innovations in Theoretical Computer Science*, ITCS ’18, pages 28:1–28:20, Dagstuhl, Germany, 2018. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.
- Wenlong Mou, Zheng Wen, and Xi Chen. On the sample complexity of reinforcement learning with policy space generalization. *arXiv preprint arXiv:2008.07353*, 2020.
- J Ian Munro and Mike S Paterson. Selection and sorting with limited storage. *Theoretical computer science*, 12(3):315–323, 1980.
- Shanmugavelayutham Muthukrishnan et al. Data streams: Algorithms and applications. *Foundations and Trends® in Theoretical Computer Science*, 1(2):117–236, 2005.

- Moni Naor and Vanessa Teague. Anti-persistence: History independent data structures. In *Proceedings of the thirty-third annual ACM symposium on Theory of computing*, pages 492–501, 2001.
- Moni Naor, Gil Segev, and Udi Wieder. History-independent cuckoo hashing. In *Automata, Languages and Programming: 35th International Colloquium, ICALP 2008, Reykjavik, Iceland, July 7-11, 2008, Proceedings, Part II 35*, pages 631–642. Springer, 2008.
- Seth Neel, Aaron Roth, and Saeed Sharifi-Malvajerdi. Descent-to-delete: Gradient-based methods for machine unlearning. In *Proceedings of the 32nd International Conference on Algorithmic Learning Theory, ALT '21*. JMLR, Inc., 2021.
- Quoc Phong Nguyen, Bryan Kian Hsiang Low, and Patrick Jaillet. Variational Bayesian unlearning. In *NeurIPS*, pages 16025–16036, 2020.
- Thanh Tam Nguyen, Thanh Trung Huynh, Phi Le Nguyen, Alan Wee-Chung Liew, Hongzhi Yin, and Quoc Viet Hung Nguyen. A survey of machine unlearning. *arXiv:2209.02299*, 2022.
- Martin Pawelczyk, Seth Neel, and Himabindu Lakkaraju. In-context unlearning: Language models as few shot unlearners. *arXiv preprint arXiv:2310.07579*, 2023.
- Martin Pawelczyk, Jimmy Z Di, Yiwei Lu, Gautam Kamath, Ayush Sekhari, and Seth Neel. Machine unlearning fails to remove data poisoning attacks. *arXiv preprint arXiv:2406.17216*, 2024.
- Binghui Peng and Aviad Rubinstein. Near optimal memory-regret tradeoff for online learning. In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1171–1194. IEEE, 2023.
- Jeff M Phillips and Jeff M Phillips. Big data and sketching. *Mathematical Foundations for Data Analysis*, pages 261–281, 2021.
- Shauli Ravfogel, Michael Twiton, Yoav Goldberg, and Ryan D Cotterell. Linear adversarial concept erasure. In *International Conference on Machine Learning*, pages 18400–18421. PMLR, 2022a.
- Shauli Ravfogel, Francisco Vargas, Yoav Goldberg, and Ryan Cotterell. Adversarial concept erasure in kernel space. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6034–6055, 2022b.
- Ran Raz. Fast learning requires good memory: A time-space lower bound for parity learning. In *Proceedings of the 57th Annual IEEE Symposium on Foundations of Computer Science, FOCS '16*, pages 266–275. IEEE Computer Society, 2016.
- Enrique Romero, Ignacio Barrio, and Lluís Belanche. Incremental and decremental learning for linear support vector machines. In *ICANN*, pages 209–218, 2007.
- Daniel Russo and Benjamin Van Roy. Eluder dimension and the sample complexity of optimistic exploration. *Advances in Neural Information Processing Systems*, 26, 2013.
- Ayush Sekhari, Jayadev Acharya, Gautam Kamath, and Ananda Theertha Suresh. Remember what you want to forget: Algorithms for machine unlearning. In *NeurIPS*, pages 18075–18086, 2021.

- Ohad Shamir. Fundamental limits of online and distributed algorithms for statistical learning and estimation. *Advances in Neural Information Processing Systems*, 27, 2014.
- Vatsal Sharan, Aaron Sidford, and Gregory Valiant. Memory-sample tradeoffs for linear regression with small error. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, pages 890–901, 2019.
- Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *S & P*, pages 3–18, 2017.
- Vaidehi Srinivas, David P Woodruff, Ziyu Xu, and Samson Zhou. Memory bounds for the experts problem. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, pages 1158–1171, 2022.
- Jacob Steinhardt and John Duchi. Minimax rates for memory-bounded sparse linear regression. In *Conference on Learning Theory*, pages 1564–1587. PMLR, 2015.
- Jacob Steinhardt, Gregory Valiant, and Stefan Wager. Memory, communication, and statistical queries. In *Conference on Learning Theory*, pages 1490–1516. PMLR, 2016.
- Vinith M Suriyakumar and Ashia C Wilson. Algorithms that approximate data removal: New results and limitations. In *NeurIPS*, 2022.
- Anvith Thudi, Gabriel Deza, Varun Chandrasekaran, and Nicolas Papernot. Unrolling SGD: Understanding factors influencing machine unlearning. In *EuroS&P*, pages 303–319, 2022.
- Amund Tveit, Magnus Lie Hetland, and Håavard Engum. Incremental and decremental proximal support vector classification using decay coefficients. In *DaWak*, pages 422–429, 2003.
- Vladimir N Vapnik. The nature of statistical learning theory, 1995.
- Lingzhi Wang, Tong Chen, Wei Yuan, Xingshan Zeng, Kam-Fai Wong, and Hongzhi Yin. Kga: A general machine unlearning framework based on knowledge gap alignment. *arXiv preprint arXiv:2305.06535*, 2023.
- Blake Woodworth and Nathan Srebro. Open problem: The oracle complexity of convex optimization with limited memory. In *Conference on Learning Theory*, pages 3202–3210. PMLR, 2019.
- Yinjun Wu, Edgar Dobriban, and Susan Davidson. Deltagrad: Rapid retraining of machine learning models. In *International Conference on Machine Learning (ICML)*, 2020.
- Santiago Zanella-Béguelin, Lukas Wutschitz, Shruti Tople, Victor Rühle, Andrew Paverd, Olga Ohrimenko, Boris Köpf, and Marc Brockschmidt. Analyzing information leakage of updates to natural language models. In *CCS*, pages 363–375, 2020.
- Rui Zhang and Shihua Zhang. Rethinking influence functions of neural networks in the over-parameterized regime. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2021.
- Ruiqi Zhang, Licong Lin, Yu Bai, and Song Mei. Negative preference optimization: From catastrophic collapse to effective unlearning. *arXiv preprint arXiv:2404.05868*, 2024.

Contents of Appendix

A	Discussion of Related Works	21
B	Additional Preliminaries	23
B.1	Combinatorial Dimensions	23
B.2	Examples of Simple Inefficient LU Schemes	23
B.3	Information Theoretic Notions	23
C	Missing Details from Section 3	24
C.1	Proof of Theorem 5	24
C.2	Proof of Theorem 7	25
D	Missing Details from Section 4	28
D.1	Proof of Theorem 10	28
D.2	Proof of Theorem 15	29
D.3	Proof of Lemma 17	30
D.4	Proof of Theorem 18	31
D.5	Proof of Lemma 20	32
D.6	Proof of Theorem 13	33
D.7	Missing Details from Section 4.2	34
D.8	Version-space Compression is Necessary for Unlearning in the Central Model	35
E	Missing Details from Section 5	36
F	Learning-Unlearning Schemes for Halfspaces	37
G	Relation Between Various Combinatorial Dimensions	41
H	From Realizability Testing to Unlearning	43
H.1	A Reduction from Unlearning Schemes for ERM to Realizability Testing	46
H.1.1	Unifying Lower Bounds for Unlearning Schemes for Realizability Testing	46
H.1.2	The White-Box Reduction	47

Appendix A. Discussion of Related Works

Machine unlearning and the “Right to be Forgotten”. Over the past few years, there has been tremendous research on formalizing the notion of “right to be forgotten” and “data deletion” under different circumstances (Dukler et al., 2023; Krishna et al., 2023; Cohen et al., 2023; Eisenhofer et al., 2022; Garg et al., 2020; Cohen and Nissim, 2020; Godin and Lamontagne, 2021). Taking inspiration from cryptography, Garg et al. (2020) define the notion of deletion-compliance, using a “leave no trace” approach, such that post deletion, the state of the world should be indistinguishable from the state of the world if the data point was never in the data set at all. Cohen et al. (2023) define a notion called deletion-as-control which conceptually relaxes the notion of deletion-compliance by only requiring the effects of the data that requested to be deleted to be absent from the updated model. Our unlearning setting is closer to that of Cohen et al. (2023).

In terms of the objectives of data deletion, many works focus on removing the influence of a particular subset of training data points (Ginart et al., 2019; Wu et al., 2020; Golatkar et al., 2020a,b; Bourtole et al., 2021; Izzo et al., 2021; Neel et al., 2021; Sekhari et al., 2021; Jang et al., 2022; Huang and Canonne, 2023; Wang et al., 2023), while others try to remove specific concepts (Ravfogel et al., 2022a,b; Belrose et al., 2023), or corrupted/poisoned samples (Goel et al., 2024; Pawelczyk et al., 2024). A detailed comparison between different definitions and objectives can be found in Cohen et al. (2023).

Reducing the cost of retraining. There is a vast literature on algorithms and techniques that can be unlearned without incurring the cost of retraining from scratch. On one hand, we have rigorous methods for exact unlearning (Bourtole et al., 2021; Thudi et al., 2022), that exactly recover the retrained-from-scratch hypothesis; such methods are typically quite memory intensive and store multiple models trained on different subsets of the training dataset. On the other hand, there are many algorithms for approximate unlearning (Ginart et al., 2019; Sekhari et al., 2021; Neel et al., 2021; Gupta et al., 2021; Chourasia et al., 2023), where the goal is to recover the retrained-from-scratch hypothesis probabilistically under a definition inspired from differential privacy (Dwork et al., 2006); such algorithms are generally space efficient but are restricted to specialized classes like convex models (Guo et al., 2020; Sekhari et al., 2021; Neel et al., 2021; Suriyakumar and Wilson, 2022; Huang and Canonne, 2023), linear regression (Cook and Weisberg, 1980; Guo et al., 2020; Izzo et al., 2021), support vector machines (Cauwenberghs and Poggio, 2000; Tveit et al., 2003; Karasuyama and Takeuchi, 2010; Romero et al., 2007), kernel methods (Zhang and Zhang, 2021), random forests (Brophy and Lowd, 2021), etc. In addition to these theoretically rigorous works, there are also many empirical approaches, that theoretical guarantees but are implementable for various large scale deep learning settings (Du et al., 2019; Golatkar et al., 2020a; Goel et al., 2022; Kurmanji et al., 2023; Ravfogel et al., 2022a,b; Belrose et al., 2023; Du et al., 2019; Golatkar et al., 2021; Nguyen et al., 2020; Graves et al., 2021; Zanella-Béguelin et al., 2020; Eldan and Russinovich, 2023; Pawelczyk et al., 2023; Zhang et al., 2024).

Learning and unlearning under memory constraints. A few recent works have considered the memory footprint of machine unlearning, and the associated tradeoffs due to memory constraints (Sekhari et al., 2021; Ghazi et al., 2023). Ghazi et al. (2023) introduced the notion of central and ticketed learning-unlearning schemes, and mergeable hypothesis classes, and provided space-efficient unlearning schemes for various specific hypothesis classes. Building on their work, our

work provides a characterization of the memory complexity of unlearning in terms of natural combinatorial notions of a hypothesis class.

The memory complexity of learning and hypothesis testing problems have been studied for several decades in the statistics and information theory communities (Cover, 1969; Hellman, 1969; Cover et al., 1976; Cover and Hellman, 1970). There was a resurgence in interest in the memory complexity of learning in the past decade (Shamir, 2014; Steinhardt and Duchi, 2015; Steinhardt et al., 2016), leading to a breakthrough result of Raz (2016), who showed that learning parities needs either quadratic memory or exponentially many samples. Subsequently, these infeasibility results under memory constraints have been extended to various problems in learning (Moshkovitz and Moshkovitz, 2017; Kol et al., 2017; Moshkovitz and Moshkovitz, 2018; Garg et al., 2018; Beame et al., 2018; Srinivas et al., 2022; Peng and Rubinstein, 2023) and optimization (Woodworth and Srebro (2019); Sharan et al. (2019); Marsden et al. (2022); Blanchard (2024)).

Connection to history independent data structures. A more traditional concept that is related to notion of machine unlearning is *history independent data structures* (Naor and Teague, 2001; Bender et al., 2024; Blleloch and Golovin, 2007; Naor et al., 2008; Hartline et al., 2005; Attiya et al., 2024; Buchbinder and Petrank, 2003). At an intuitive level, our notion of a learning-unlearning scheme can be seen as enforcing history independence between the state obtained by first adding samples followed by removal, and recomputing the data structure from scratch. However, we are not aware of any direct applications of the existing literature on history independence towards machine unlearning.

Connection to privacy. There is a rich literature in ML on protecting privacy of individuals, with the most popular notion being that of Differential Privacy (DP) (Dwork et al., 2006). Under DP, our objective is to develop learning algorithms whose output model is probabilistically indistinguishable from a model that we would have obtained by training without a particular data point. Thus, DP implies machine unlearning in the sense that differentially private learning guarantees that any single data point is already forgotten from the trained model. However, one can hope to do better since DP guarantees privacy for all points simultaneously, whereas in machine unlearning the updated hypothesis may depend on the samples that requested to be removed (Sekhari et al., 2021). That being said, the connection between DP and machine unlearning remains largely unexplored, with only a few recent works (Gupta et al., 2021; Chien et al., 2024).

Streaming algorithms and data deletion. Another related line of work is on streaming algorithms which process data samples one at a time while only using a small amount of memory. Starting from the early works of Munro and Paterson (1980); Flajolet and Martin (1985); Misra and Gries (1982); Alon et al. (1996), streaming algorithms have become an rich and extensive area under which several problems have been studied (See Muthukrishnan et al. (2005); McGregor (2014); Phillips and Phillips (2021) and references therein). Though conceptually related to the setting we are considering, we are not aware of any direct application of techniques in this setting to the problems considered in our paper. One concrete differences comes from the fact we restrict ourselves to a one-shot unlearning setting, where all the deletion requests arrive at the same time and there is a single round of unlearning. It would be interesting to extend our techniques to streaming deletion requests, e.g. as considered in (Gupta et al., 2021; Chien et al., 2024).

Appendix B. Additional Preliminaries

B.1. Combinatorial Dimensions

We start with a formal definition of the VC dimension.

Definition 25 (VC dimension Vapnik (1995)) *A set of points $\tilde{\mathcal{X}} \subseteq \mathcal{X}$ is said to be shattered if every possible signing on the points can be realized across hypotheses in $\mathcal{H}$ i.e. for all functions $t : \tilde{\mathcal{X}} \rightarrow \{0, 1\}$, there exists a function $h \in \mathcal{H}$ such that for all $x \in \tilde{\mathcal{X}}$, $t(x) = h(x)$. The VC dimension of a hypothesis class $\mathcal{H}$, denoted by $\text{VC}(\mathcal{H})$, is the cardinality of the largest subset of its domain which is shattered by the hypothesis class.*

We next define the Littlestone dimension of a hypothesis class.

Definition 26 (Littlestone dimension Littlestone (1988)) *Given a hypothesis class $\mathcal{H}$ that maps elements from a domain $\mathcal{X}$ to $\{0, 1\}$, the littlestone tree is a rooted binary tree, where each node is labeled by a point $x \in \mathcal{X}$, the left edge is labeled by $y = 0$, and the right edge is labeled by $y = 1$. The tree is said to be shattered by $\mathcal{H}$ if, for every branch in the tree there exists a concept $h \in \mathcal{H}$ which is consistent with the edges of the branch (i.e. for every x and the decision y on this branch, we have $h(x) = y$).*

The Littlestone dimension of the hypothesis class $\mathcal{H}$ is the largest $\ell \in \mathbb{N} \cap \{0\}$, for which there exists a complete Littlestone tree of depth ℓ all of whose branches are shattered by $\mathcal{H}$.

B.2. Examples of Simple Inefficient LU Schemes

When the space complexity is allowed to scale linearly with any of these parameters, a learner can use the following trivial strategies to implement a central-memory $(0, 0)$ -LU scheme.

- The learner stores the entire training dataset in aux. For any unlearning query, the unlearner recomputes the solution to the learning task on the remainder of the dataset. This requires $O(n \log |\mathcal{X}|)$ bits of storage in the central memory.
- For every $x \in \mathcal{X}$, the learner stores how many times $(x, 0)$ and $(x, 1)$ appear in the dataset. For any unlearning query, the unlearner can again deduce the remaining dataset. This requires $O(|\mathcal{X}| \log n)$ bits of storage.
- For every hypothesis $h \in \mathcal{H}$, the learner stores the count of data points that disagree with h . For any unlearning query, the unlearner recomputes this count for every hypothesis, and checks if any count is 0. This requires $O(|\mathcal{H}| \log n)$ bits of storage.

B.3. Information Theoretic Notions

We will use information theoretic quantities to prove the space complexity lower bounds. We use capital bold letters such $\mathbf{X}, \mathbf{Y}, \mathbf{Z}$, etc., to denote random variables and x, y, z , etc., to denote the values these random variables take. Given a probability distribution $\mathbf{P} : \mathcal{X} \rightarrow [0, 1]$, we use the notation $x \sim \mathbf{P}$ when value x is sampled according to distribution $\mathbf{P}$. Similarly, we use the notation $z \sim \mathbf{Z}$ to denote the process that $\mathbf{Z}$ takes value z with probability $\Pr[\mathbf{Z} = z]$. We use $\text{Ber}(q)$ to denote the Bernoulli distribution which takes value 1 with probability q and 0 with probability $1 - q$. We use notations $\mathbb{E}[\mathbf{Z}]$ to denote the expectation of random variable $\mathbf{Z}$. $\mathbb{E}[\mathbf{Z} \mid \mathbf{Y} = y]$ denote the expectation of random variable $\mathbf{Z}$ conditioned on the event $\mathbf{Y} = y$.

Basics of Information Theory. Given a random variable $\mathbf{Z}$, $H(\mathbf{Z})$ denotes the Shannon entropy of $\mathbf{Z}$, that is, $H(\mathbf{Z}) = \sum_z \Pr(\mathbf{Z} = z) \log_2(1/\Pr(\mathbf{Z} = z))$. We also use $H(\mathbf{P})$ to denote the entropy of a probability distribution $\mathbf{P}$. Overloading the notation, we will denote the entropy of $\text{Ber}(q)$ by $H(q)$.

$I(\mathbf{X}; \mathbf{Y} \mid \mathbf{Z})$ represents the mutual information between $\mathbf{X}$ and $\mathbf{Y}$ conditioned on the random variable $\mathbf{Z}$. $I(\mathbf{X}; \mathbf{Y} \mid \mathbf{Z}) = H(\mathbf{X} \mid \mathbf{Z}) - H(\mathbf{X} \mid \mathbf{Y}, \mathbf{Z})$, where $H(\mathbf{X} \mid \mathbf{Y}) = \mathbb{E}_{y \sim \mathbf{Y}} H(\mathbf{X} \mid \mathbf{Y} = y) \leq H(\mathbf{X})$. Next, we describe some of the properties of mutual information used in the paper.

1. (Chain Rule) $I(\mathbf{X}; \mathbf{Y}, \mathbf{Z}) = I(\mathbf{X}; \mathbf{Y}) + I(\mathbf{X}; \mathbf{Z} \mid \mathbf{Y})$.
2. If $I(\mathbf{X}; \mathbf{Y}) = 0$, then $I(\mathbf{X}; \mathbf{Z} \mid \mathbf{Y}) \geq I(\mathbf{X}; \mathbf{Z})$.
3. If $I(\mathbf{X}; \mathbf{Y} \mid \mathbf{Z}) = 0$, then $I(\mathbf{X}; \mathbf{Z}) \geq I(\mathbf{X}; \mathbf{Z} \mid \mathbf{Y})$.

Property 1 follows from the chain rule for Shannon entropy (H). Properties 2 and 3 follow from the observation that

$$I(\mathbf{X}; \mathbf{Y} \mid \mathbf{Z}) + I(\mathbf{X}; \mathbf{Z}) = I(\mathbf{X}; \mathbf{Y}, \mathbf{Z}) = I(\mathbf{X}; \mathbf{Y}) + I(\mathbf{X}; \mathbf{Z} \mid \mathbf{Y}).$$

As mutual information is non-negative, if $I(\mathbf{X}; \mathbf{Y} \mid \mathbf{Z}) = 0$, then $I(\mathbf{X}; \mathbf{Z}) \geq I(\mathbf{X}; \mathbf{Z} \mid \mathbf{Y})$ and if $I(\mathbf{X}; \mathbf{Y}) = 0$, then $I(\mathbf{X}; \mathbf{Z} \mid \mathbf{Y}) \geq I(\mathbf{X}; \mathbf{Z})$.

Appendix C. Missing Details from Section 3

C.1. Proof of Theorem 5

Proof of Theorem 5. Let $d = \text{VC}(\mathcal{H})$. Consider the collection of datasets $\mathcal{D} = \{D_z : z \in \{0, 1\}^d\}$; for any maximal set of points shattered by $\mathcal{H}$, $\{x_1, \dots, x_d\}$, D_z is constructed by placing two coincident points on each x_i , one labeled 1 and the other labeled as z_i .

Unlearning queries. For all $i \in [d]$, an unlearning query U_i is a collection of points $\{(x_j, 1) : j \in [d] \setminus \{i\}\}$. Namely, one copy of all the points with label 1 are removed, except for at the point x_i . It is important to note that the points being deleted in the unlearning query are not a function of the z_i 's which are a priori unknown. For every starting dataset D_z , since $\{x_1, \dots, x_d\}$ is a shattered set, the surviving dataset is $\mathcal{H}$ -realizable if and only if there are no coincident points with opposite labels. Under the unlearning query U_i , this is possible only if $z_i = 1$. Thus, the outputs of the **Unlearn** algorithm on these queries can be used to infer information about the starting dataset.

Formally, let $\mathbf{Z}$ be a random variable with uniform distribution over $\{0, 1\}^d$. Let $\mathbf{D}$ be a random variable over datasets, taking value D_z when $\mathbf{Z} = z$. Let $(\text{Learn}, \text{Unlearn})$ be an (ε, δ) -LU scheme for $\mathcal{H}$ -realizability testing. As for the learning task of realizability testing, $\mathcal{W} = \{\text{Yes}, \text{No}\}$, $\text{Learn}(D)$ always outputs **Yes** if D is $\mathcal{H}$ -realizable and **No** otherwise. Let $\mathbf{Aux}$ be a random variable for the auxiliary information outputted by **Learn** algorithm on dataset $\mathbf{D}$. By Definition 3, for all datasets $D_z \in \mathcal{D}$ and unlearning queries $\{U_i\}_{i \in [d]}$, if $D_z \setminus U_i$ is realizable $- z_i = 1$, then

$$\Pr[\text{Unlearn}(U_i, \text{aux}(D_z)) = \text{No}] \leq e^\varepsilon \cdot 0 + \delta = \delta.$$

Similarly, if $D_z \setminus U_i$ is not realizable $- z_i = 0$, then

$$\Pr[\text{Unlearn}(U_i, \text{aux}(D_z)) = \text{Yes}] \leq \delta.$$



Figure 1: Lower bound for VC classes

Using Bayes' rule, it's easy to see that probability of $z_i = 1$ (or 0) conditioned on $\text{Unlearn}(U_i, \text{aux}(D_z)) = \text{Yes}$ (or No) is greater than $1 - \delta$. Thus, for $\delta < 1/2$,

$$H(\mathbf{Z}_i \mid \text{Unlearn}(U_i, \mathbf{Aux})) \leq H(1 - \delta) \implies I(\mathbf{Z}_i; \text{Unlearn}(U_i, \mathbf{Aux})) \geq (1 - H(1 - \delta)). \quad (1)$$

We will use this to show that $\mathbf{Aux}$ contains $\Omega(1)$ bits of information about $\mathbf{Z}_i$ for all $i \in [d]$, and must use $\Omega(d)$ bits of space. We use $|\mathbf{Aux}|$ to denote the size of auxiliary information on datasets in $\mathcal{D}$. Formally,

$$\begin{aligned} |\mathbf{Aux}| &\geq I(\mathbf{Z}; \mathbf{Aux}) && (\text{as } H(\mathbf{Aux}) \leq |\mathbf{Aux}|) \\ &= \sum_{i=1}^d I(\mathbf{Z}_i; \mathbf{Aux} \mid \mathbf{Z}_{<i}) && (\text{Chain rule}) \\ &\geq \sum_{i=1}^d I(\mathbf{Z}_i; \mathbf{Aux}) && (\text{as } I(\mathbf{Z}_i; \mathbf{Z}_{<i}) = 0) \\ &= \sum_{i=1}^d (I(\mathbf{Z}_i; \mathbf{Aux}, \text{Unlearn}(U_i, \mathbf{Aux})) - I(\mathbf{Z}_i; \text{Unlearn}(U_i, \mathbf{Aux}) \mid \mathbf{Aux})) \\ &\geq \sum_{i=1}^d (I(\mathbf{Z}_i; \text{Unlearn}(U_i, \mathbf{Aux})) - I(\mathbf{Z}_i; \text{Unlearn}(U_i, \mathbf{Aux}) \mid \mathbf{Aux})) \\ &\geq \sum_{i=1}^d (1 - H(1 - \delta)) && (\text{Explained below}) \\ &= d \cdot (1 - H(\delta)) \end{aligned}$$

The second last inequality follows from Equation (1) and the fact that $I(\mathbf{Z}_i; \text{Unlearn}(U_i, \mathbf{Aux}) \mid \mathbf{Aux}) = 0$. This is because $\text{Unlearn}(U_i, \mathbf{Aux})$ is completely determined by $\mathbf{Aux}$ and the random bits used by the Unlearn algorithm, which are independent of $\mathbf{Z}_i$ (and $\mathbf{Aux}$). Hence, any (ϵ, δ) -LU scheme must have space complexity of at least $d(1 - H(\delta))$ bits on datasets of size at most $2d$. ■

C.2. Proof of Theorem 7

Proof of Theorem 7. We will construct the lower bound by considering a collection of datasets and arguing that any LU/ TiLU scheme which supports unlearning queries on $1/\epsilon$ points can be used to essentially infer the entire dataset. For the ticketed lower bound, we furthermore show that it suffices to select the $1/\epsilon$ -sized unlearning query from a small subset of $\approx n^\epsilon/\epsilon$ points. In particular, the total information contained within the tickets of these few points, put together, can recover the

entire dataset, which will imply that the maximum ticket size will have to be large.

Dataset. The collection of datasets we consider is denoted $\mathcal{D} = \{D(z) : z \in \{0, 1\}^m\}$ and illustrated in Figure 1. All of the datasets in $\mathcal{D}$ contain a fixed collection of m^ϵ/ϵ points denoted S_{code} , all labelled 0. As a function of the hidden vector $z \in \{0, 1\}^m$, the dataset contains additional points. If $z_i = 1$, the dataset contains an additional point x_i labelled 0. The total number of points contained in datasets in $\mathcal{D}$ are between m^ϵ/ϵ and $m + m^\epsilon/\epsilon$ which are both $O(m)$ for constant $\epsilon \in (0, 1]$.

Hypothesis class. The hypothesis class $\mathcal{H}$ we consider has m functions, $\{h_i : i \in [m]\}$. The function h_i , satisfies the following properties,

- (i) For any $j \neq i$, h_i labels x_j as 0.
- (ii) For $j = i$, h_i labels x_i as 1.
- (iii) For the points in S_{code} , consider any injective function σ from $[m] \rightarrow \binom{S_{\text{code}}}{1/\epsilon}$, the $1/\epsilon$ -sized subsets of S_{code} . Such an injective function exists since $|S_{\text{code}}| = m^\epsilon/\epsilon$ and therefore the size of the co-domain $\binom{m^\epsilon/\epsilon}{1/\epsilon} \geq m$ is larger than that of the domain. h_i labels the points in S_{code} as 0 except for all the points in $\sigma(i)$, which are labelled as 1.

Littlestone dimension of $\mathcal{H}$. The Littlestone dimension of $\mathcal{H}$ is at most $1/\epsilon + 1$. Consider the online learning algorithm which labels every point in the dataset with the label 0. Notice that any labeling of the dataset consistent with some hypothesis $h \in \mathcal{H}_i$ for some i , labels at most $1/\epsilon + 1$ points as 1. This means that the online learning algorithm we consider makes at most $1/\epsilon + 1$. The Littlestone dimension of the class, which captures the optimal mistake bound of realizable online learning for the class is therefore at most $1/\epsilon + 1$.

Unlearning queries. The unlearning queries we consider correspond to choosing an arbitrary subset of $1/\epsilon$ points from S_{code} . The learner iterates over all such subsets to get the set of unlearning queries.

Unlearning $\implies z$ can be recovered. Consider an unlearning query U_i of $1/\epsilon$ points from S_{code} corresponding to $\sigma(i)$ for some $i \in [m]$. The hypothesis h_i is potentially the only one which can label all the remaining points in S_{code} correctly. The only points labeled incorrectly by h_i is potentially the point x_i . In particular, since h_i labels this point as 1, after the unlearning query U_i is made, the dataset is $\mathcal{H}$ -realizable if and only if x_i did not exist in it to begin with. In particular, under the unlearning query U_i , the realizability of the dataset answers whether $z_i = 0$ or $z_i = 1$. Therefore, the hidden vector $(z_1, \dots, z_m)$ can be recovered by considering all such unlearning queries $\{U_i : i \in [m]\}$.

We now formalize this argument. Let $\mathbf{Z}$ be a random variable with uniform distribution over $\{0, 1\}^m$. Let $\mathbf{D}$ be a random variable over datasets, taking value D_z when $\mathbf{Z} = z$. Let $(\text{Learn}, \text{Unlearn})$ be an (ϵ, δ) -LU scheme for $\mathcal{H}$ -realizability testing. As for the learning task of realizability testing, $\mathcal{W} = \{\text{Yes}, \text{No}\}$, $\text{Learn}(D)$ always outputs **Yes** if D is $\mathcal{H}$ -realizable and **No** otherwise. Let $\mathbf{Aux}$ be a random variable for the auxiliary information outputted by **Learn** algorithm on dataset $\mathbf{D}$. By Definition 3, for all datasets $D_z \in \mathcal{D}$ and unlearning queries $\{U_i\}_{i \in [m]}$, if $D_z \setminus U_i$ is realizable – $z_i = 1$, then

$$\Pr[\text{Unlearn}(U_i, \text{aux}(D_z)) = \text{No}] \leq e^\epsilon \cdot 0 + \delta = \delta. \quad (2)$$

Similarly, if $D_z \setminus U_i$ is not realizable $- z_i = 0$, then

$$\Pr[\text{Unlearn}(U_i, \text{aux}(D_z)) = \text{Yes}] \leq \delta. \quad (3)$$

Thus, using Bayes' rule, it's easy to see that probability of $z_i = 1$ (or 0) conditioned on $\text{Unlearn}(U_i, \text{aux}(D_z)) = \text{Yes}$ (or No) is greater than $1 - \delta$. Thus, for $\delta < 1/2$,

$$H(\mathbf{Z}_i \mid \text{Unlearn}(U_i, \mathbf{Aux})) \leq H(1 - \delta) \implies H(\mathbf{Z}_i \mid \mathbf{Aux}) \leq H(1 - \delta), \quad (4)$$

where the second implication follows from data processing inequality. Thus, $\mathbf{Aux}$ contains $\Omega(1)$ bits of information about $\mathbf{Z}_i$ for all $i \in [m]$, and must use $\Omega(m)$ bits of space. Formally,

$$\begin{aligned} |\mathbf{Aux}| &\geq I(\mathbf{Z}; \mathbf{Aux}) && (\text{as } H(\mathbf{Aux}) \leq |\mathbf{Aux}|) \\ &= \sum_{i=1}^m I(\mathbf{Z}_i; \mathbf{Aux} \mid \mathbf{Z}_{<i}) && (\text{Chain rule}) \\ &\geq \sum_{i=1}^m I(\mathbf{Z}_i; \mathbf{Aux}) && (\text{as } I(\mathbf{Z}_i; \mathbf{Z}_{<i}) = 0) \\ &= \sum_{i=1}^m (H(\mathbf{Z}_i) - H(\mathbf{Z}_i \mid \mathbf{Aux})) \\ &\geq \sum_{i=1}^m (1 - H(1 - \delta)) \\ &= m \cdot (1 - H(\delta)) \end{aligned}$$

This implies that any (ε, δ) -LU scheme must have space complexity of at least $m(1 - H(\delta))$ bits on datasets of size at most $m + m^\beta/\beta$.

Moving on the TiLU scheme, the lower bound is obtained by considering an additional argument. Note that the unlearning queries considered are only of points in S_{code} , which are m^β/β in number. In particular, by collecting the tickets of all the points in S_{code} , the TiLU scheme should have enough information to recover the entire hidden vector $z \in \{0, 1\}^d$. Formally, introduce the random variable $\mathbf{Tix}$ which is the collection of tickets of all the points in S_{code} .

$$\begin{aligned} |\mathbf{Aux}| + \sum_{i \in S_{\text{code}}} |\mathbf{Tix}_i| &\geq I(\mathbf{Z}; \mathbf{Aux}, \mathbf{Tix}) \\ &\geq \sum_{i=1}^m I(\mathbf{Z}_i; \mathbf{Aux}, \mathbf{Tix} \mid \mathbf{Z}_{<i}) \\ &\geq \sum_{i=1}^m I(\mathbf{Z}_i; \mathbf{Aux}, \mathbf{Tix}) \\ &= \sum_{i=1}^m (H(\mathbf{Z}_i) - H(\mathbf{Z}_i \mid \mathbf{Aux}, \mathbf{Tix})) \\ &\geq \sum_{i=1}^m (1 - H(1 - \delta)) \\ &= m(1 - H(\delta)). \end{aligned}$$

Thus, we have that $|\mathbf{Aux}| + \sum_{i \in S_{\text{code}}} |\mathbf{Tix}_i| \geq m(1 - H(\delta))$ which implies that $\max\{|\mathbf{Aux}|, \max_i |\mathbf{Tix}_i|\} \geq \frac{m}{2|S_{\text{code}}|} = \Omega(\beta m^{1-\beta})$. Rewriting in terms of $n (= m + m^\beta/\beta)$ results in the statement of the theorem.

■

Appendix D. Missing Details from Section 4

D.1. Proof of Theorem 10

Proof of Theorem 10. Let $\{(x_i, y_i) \in \mathcal{X} \times \{0, 1\}\}_{i=1}^{\epsilon}$ denote the longest eluder sequence for the hypothesis class $\mathcal{H}$. Namely, for each $i \leq \epsilon$, there exists hypotheses $h, h' \in \mathcal{H}$ such that $h(x_j) = h'(x_j) = y_j$ for all $j < i$, but $h(x_i) \neq h'(x_i)$.

Datasets. Let $m = \min\{n/2, \epsilon\}$. Consider a collection of datasets of size at most n , $\mathcal{D} = \{D_z : z \in \{0, 1\}^m\}$. Namely, the datasets in $\mathcal{D}$ are indexed by a m -length binary string. The dataset D_z is defined with having z_i copies of the point (x_i, y_i) and one copy of the point $(x_i, \neg y_i)$. Note that the size of the datasets in $\mathcal{D}$ is at most $2m \leq n$.

Recovering z using unlearning queries. Informally, when the underlying dataset is chosen as D_z for an arbitrary $z \in \{0, 1\}^m$, by making unlearning queries, we show that z can be recovered. This means that the LU scheme must store $\Omega(m)$ bits of information. The unlearning queries we consider peel away z revealing it one coordinate at a time. Unlike the proof of Theorem 5 and 7, our unlearning queries would depend on partial information about z .

As before, let $\mathbf{Z}$ be a random variable with uniform distribution over $\{0, 1\}^m$. Let $\mathbf{D}$ be a random variable over datasets, taking value D_z when $\mathbf{Z} = z$. Let $(\text{Learn}, \text{Unlearn})$ be an (ϵ, δ) -LU scheme for $\mathcal{H}$ -realizability testing. Let $\mathbf{Aux}$ be a random variable for the auxiliary information outputted by Learn algorithm on dataset $\mathbf{D}$. For all $i \in [m]$, let $\mathbf{U}_i$ be a random variable representing an unlearning query, which is a collection of datapoints $(x_j, \neg y_j)_{j < i}$ and all data points on features $\{x_j : j > i\}$. Formally, when $\mathbf{Z} = z$, $\mathbf{U}_i = U_{i,z} = \{(x_j, \neg y_j) : j \neq i\} \cup \{(x_j, y_j) : j > i \text{ and } z_j = 1\}$. Note that $\forall i \in [m], \mathbf{U}_i \subseteq \mathbf{D}$.

Fix z and consider the dataset D_z and the unlearning query $U_{i,z}$. The i th unlearning query removes all the points labeled $(x_j, \neg y_j)$ for $j < i$ and all the points with features in $\{x_j : j > i\}$. Note that, if $z_i = 1$, the dataset remains $\mathcal{H}$ -unrealizable since there are coincident points at x_i labeled y_i and $\neg y_i$. If $z_i = 0$, then the dataset $D_z \setminus U_{i,z}$ becomes realizable, since by definition of an eluder sequence, there must be a hypothesis in $\mathcal{H}$ which agrees with all the surviving labels $\{(x_j, y_j) : j < i\}$ while labeling x_i as either 0 or 1, whichever equates to $\neg y_i$. Thus, the Learn algorithm outputs **Yes** on the dataset $D_z \setminus U_{i,z}$ if and only if $z_i = 0$. By Definition 3, for all z, i , if $D_z \setminus U_{i,z}$ is realizable – $z_i = 0$, then

$$\Pr(\text{Unlearn}(U_{i,z}, \text{aux}(D_z)) = \text{No}) \leq e^\epsilon \cdot 0 + \delta = \delta.$$

Similarly, if $D_z \setminus U_{i,z}$ is not realizable – $z_i = 1$, then

$$\Pr(\text{Unlearn}(U_{i,z}, \text{aux}(D_z)) = \text{Yes}) \leq e^\epsilon \cdot 0 + \delta = \delta.$$

Using Bayes' rule, it's easy to see that probability of $z_i = 0$ (or 1) conditioned on the output of $\text{Unlearn}(U_{i,z}, \text{aux}(D_z))$ being **Yes** (or **No**) is greater than $1 - \delta$. Thus, for $\delta < 1/2$,

$$\mathbb{H}(\mathbf{Z}_i \mid \text{Unlearn}(\mathbf{U}_i, \mathbf{Aux})) \leq \mathbb{H}(1 - \delta) \implies \mathbb{I}(\mathbf{Z}_i; \text{Unlearn}(\mathbf{U}_i, \mathbf{Aux})) \geq (1 - \mathbb{H}(1 - \delta)). \quad (5)$$

We will use this to show that $\mathbf{Aux}$ contains $\Omega(m)$ bits of information about $\mathbf{Z}$. We use $|\mathbf{Aux}|$ to denote the size of auxiliary information on datasets in $\mathcal{D}$. Formally,

$$\begin{aligned}
 |\mathbf{Aux}| &\geq I(\mathbf{Z}; \mathbf{Aux}) && (\text{as } H(\mathbf{Aux}) \leq |\mathbf{Aux}|) \\
 &= \sum_{i=1}^d I(\mathbf{Z}_i; \mathbf{Aux} \mid \mathbf{Z}_{>i}) && (\text{Chain rule}) \\
 &= \sum_{i=1}^d (I(\mathbf{Z}_i; \mathbf{Aux}, \text{Unlearn}(U_i, \mathbf{Aux}) \mid \mathbf{Z}_{>i}) - I(\mathbf{Z}_i; \text{Unlearn}(U_i, \mathbf{Aux}) \mid \mathbf{Aux}, \mathbf{Z}_{>i})) \\
 &\geq \sum_{i=1}^d (I(\mathbf{Z}_i; \text{Unlearn}(U_i, \mathbf{Aux}) \mid \mathbf{Z}_{>i}) - I(\mathbf{Z}_i; \text{Unlearn}(U_i, \mathbf{Aux}) \mid \mathbf{Aux}, \mathbf{Z}_{>i})) \\
 &= \sum_{i=1}^d I(\mathbf{Z}_i; \text{Unlearn}(U_i, \mathbf{Aux}) \mid \mathbf{Z}_{>i}) && (\text{Explained below}) \\
 &\geq \sum_{i=1}^d I(\mathbf{Z}_i; \text{Unlearn}(U_i, \mathbf{Aux})) && (\text{as } I(\mathbf{Z}_i; \mathbf{Z}_{>i}) = 0) \\
 &\geq \sum_{i=1}^d (1 - H(1 - \delta)) && (\text{Equation (5)}) \\
 &= d \cdot (1 - H(\delta))
 \end{aligned}$$

The second last equality follows from the fact that $I(\mathbf{Z}_i; \text{Unlearn}(U_i, \mathbf{Aux}) \mid \mathbf{Aux}, \mathbf{Z}_{>i}) = 0$. This is because the unlearning query U_i is deterministic conditioned on $\mathbf{Z}_{>i}$. Furthermore, the output of the `Unlearn` algorithm on this query is completely determined by $\mathbf{Aux}$ and the random bits used by the `Unlearn` algorithm, which are independent of $\mathbf{Z}_{\geq i}$ (and $\mathbf{Aux}$). Hence, any (ε, δ) -LU scheme must have space complexity of at least $m(1 - H(\delta))$ bits on datasets of size at most $2m$. The theorem follows as $m = \min(n/2, \epsilon(\mathcal{H}))$. $\blacksquare$

D.2. Proof of Theorem 15

Proof of Theorem 15. We define the hypothesis class and corresponding TiLU scheme below.

Hypothesis class. Consider the domain $\mathcal{X} = [|\mathcal{X}|]$ and the hypothesis class $\mathcal{H}$ which captures every possible labeling on $[d]$ and every $h \in \mathcal{H}$ labels every $x > d$ as 0. We show that there exists a TiLU scheme with space complexity upper bounded by $O(\max\{\log(d), \log(n), \log(|\mathcal{X}|)\})$. Consider an arbitrary data set on $\mathcal{X}$, defined by the n_x^0 points labeled 0 and the n_x^1 points labeled 1 at each $x \in \mathcal{X}$. Together, let us denote $n_x = (n_x^0, n_x^1)$.

TiLU Scheme. The TiLU scheme we consider is as follows: let S denote the sequence of $x \in [d]$, for which the dataset contains at least one copy of $(x, 0)$ and $(x, 1)$, and $x > d$ for which the dataset contains at least one copy of $(x, 1)$, with all the points arranged in increasing order. The dataset is realizable if and only if for each $x \in S$ such that $x \leq d$, either all n_x^0 copies of $(x, 0)$ or all n_x^1 copies of $(x, 1)$ are in the unlearn set, and for each $x > d$, all n_x^1 copies of $(x, 1)$ are in the unlearn set. The server stores in `aux`, $x = S_1$ and $x = S_2$ as well as n_x for both of these points. The ticket of each point $(x, y) \in D$ such that $x = S_i$ stores S_i , $n_{S_i}^0$ and $n_{S_i}^1$ as well as S_{i+1} , $n_{S_{i+1}}^0$ and $n_{S_{i+1}}^1$ (unless

$i = |S|$). The size of the tickets are therefore upper bounded by $O(\max\{\log(d), \log(n), \log(|\mathcal{X}|)\})$. The tickets of the remaining points are empty.

Correctness of the scheme. Suppose the unlearning query is composed of k_x^0 copies of $(x, 0)$ and k_x^1 copies of $(x, 1)$ being deleted for each $x \in \mathcal{X}$. The TiLU scheme answers,

- **No** if for $x = S_1$ or $x = S_2$, at least one copy of $(x, 0)$ and $(x, 1)$ survive (which can be computed since the server stores n_x for these points).
- In the complementary case, at least one point of the form (S_2, y) must have been in the unlearn set. In this case, the dataset is unrealizable (and the scheme returns **No**) unless all n_x^1 copies of $(x, 1)$ (or n_x^0 copies of $(x, 0)$, if $x \in [d]$) are in the unlearn set for $x = S_2$ and $x = S_3$ which is computable from the ticket of (S_2, y) .
- In general we may recurse this argument to consider the complimentary case where some (S_i, y) must have been in the unlearn set. In this case, the dataset is unrealizable (and the scheme returns **No**) unless all n_x^1 copies of $(x, 1)$ (or n_x^0 copies of $(x, 0)$, if $x \in [d]$) are in the unlearn set for both $x = S_i$ and $x = S_{i+1}$ which can be computed using information stored in the ticket of (S_i, y) .

Thus using information stored in the tickets, the TiLU scheme can determine whether the unlearning query covers the set of all points which must be unlearned to make the dataset realizable. ■

D.3. Proof of Lemma 17

Proof of Lemma 17. Let Enc_{VS} and Dec_{VS} be the encoding and decoding functions corresponding to the C -bit version-space compression scheme for $\mathcal{H}$. Recall that a set of hypothesis $\mathcal{H}' \subseteq \mathcal{H}$ is said to be a version space if there exists some dataset $\bar{S} \in \mathcal{Z}^*$ such that $\mathcal{H}' = \mathcal{H}(\bar{S})$. We next define a function $\text{Canonical} : 2^{\mathcal{H}} \mapsto \mathcal{Z}^*$ that maps subsets $\mathcal{H}' \subseteq \mathcal{H}$ to datasets S' such that (a) if $\mathcal{H}'$ is a version space then $\text{Canonical}(\mathcal{H}')$ returns a canonical dataset S' such that $\mathcal{H}' = \mathcal{H}(S')$, and (b) if $\mathcal{H}'$ is not a version space then $\text{Canonical}(\mathcal{H}') = \perp$. Note that for any dataset S , we have that $\mathcal{H}(S) = \mathcal{H}(S')$ for $S' = \text{Canonical}(\mathcal{H}(S))$. For the sake of completeness, we provide an implementation of Canonical function in Appendix D.7.

We next provide an implementation of the Encode, Decode and Merge methods.

Encode: Given an input S , check if S is realizable via the class $\mathcal{H}$, and

- If S is not realizable, let $S' = \text{Canonical}(\emptyset)$.
- If S is realizable, then, compute the version space $\mathcal{H}' = \{h \in \mathcal{H} \mid h(x) = y \text{ for all } (x, y) \in S\}$, and let $S' = \text{Canonical}(\mathcal{H}')$.

Return $\text{Encode}(S) = \text{Enc}_{\text{VS}}(S')$.

Decode: Given an encoding E compute the version space $\mathcal{H}' \leftarrow \text{Dec}_{\text{VS}}(E)$, and return **No** if $\mathcal{H}' = \emptyset$, and **Yes** otherwise.

Merge: Given encodings E_1 and E_2 , compute the version spaces $\mathcal{H}_1 \leftarrow \text{Dec}_{\text{VS}}(E_1)$ and $\mathcal{H}_2 \leftarrow \text{Dec}_{\text{VS}}(E_2)$, and define $\mathcal{H}'' = \mathcal{H}_1 \cap \mathcal{H}_2$. Compute the dataset $\mathcal{S}'' = \text{Canonical}(\mathcal{H}'')$ and return $\text{Merge}(E_1, E_2) = \text{Enc}_{\text{VS}}(\mathcal{S}'')$.

Showing that $\mathcal{H}$ is a mergeable hypothesis class. We next prove that the above defined procedures satisfy [Definition 12](#). Let E be the output of **Encode** for a dataset $\mathcal{S}$. By definition of the **Canonical** function, we have that $\text{Dec}_{\text{VS}}(E) = \mathcal{H}(\mathcal{S}') = \mathcal{H}' = \mathcal{H}(\mathcal{S})$. Thus, $\text{Decode}(\text{Encode}(\mathcal{S})) = \text{Yes}$ iff $\mathcal{S}$ is realizable.

We next establish the mergeability property. Note that for any sets $\mathcal{S}_1$ and $\mathcal{S}_2$, the output of $\text{Encode}(\mathcal{S}_1 \cup \mathcal{S}_2) = \text{Enc}_{\text{VS}}(\mathcal{S}')$ where $\mathcal{S}' = \text{Canonical}(\mathcal{H}(\mathcal{S}_1 \cup \mathcal{S}_2))$. On the other hand, we compute $\text{Merge}(\text{Encode}(\mathcal{S}_1), \text{Encode}(\mathcal{S}_2)) = \text{Enc}_{\text{VS}}(\mathcal{S}'')$ where $\mathcal{S}'' = \text{Canonical}(\mathcal{H}(\mathcal{S}_1) \cap \mathcal{H}(\mathcal{S}_2))$. However, since $\mathcal{H}(\mathcal{S}_1) \cap \mathcal{H}(\mathcal{S}_2) = \mathcal{H}(\mathcal{S}_1 \cup \mathcal{S}_2)$ by definition of the version spaces, we have that $\mathcal{S}' = \mathcal{S}''$ and thus $\text{Merge}(\text{Encode}(\mathcal{S}_1), \text{Encode}(\mathcal{S}_2)) = \text{Encode}(\mathcal{S}_1 \cup \mathcal{S}_2)$. Since all generated encodings are at most C -bits, we immediately get that $\mathcal{H}$ is C -bit mergeable. $\blacksquare$

D.4. Proof of [Theorem 18](#)

Proof of [Theorem 18](#). We first define the **Enc** and **Dec** methods, and then show that they satisfy [Definition 16](#) with $C = O(\epsilon(\mathcal{H}) \log(|\mathcal{Z}|))$.

Enc: Given a set of samples $\mathcal{S}$, the encoding function creates an eluder subsequence $\mathcal{S}'$ such that $\mathcal{H}(\mathcal{S}') = \mathcal{H}(\mathcal{S})$. Initialize $\mathcal{S}' = \emptyset$. Iterate over samples $(x, y) \in \mathcal{S}$:

- Let $\mathcal{H}(\mathcal{S}')$ be the set of all the hypothesis that are consistent with the samples in the set $\mathcal{S}'$ so far.
- Update $\mathcal{S}' = \mathcal{S}' \cup \{(x, y)\}$ if there exists two hypothesis $h_1, h_2 \in \mathcal{H}(\mathcal{S}')$ such that $h_1(x) \neq h_2(x)$.
- Otherwise, we are in the case that for all $h \in \mathcal{H}(\mathcal{S}')$, $h(x) = b$ for some $b \in \{0, 1\}$. If $b = y$, then go to the next sample. Otherwise, add (x, y) to $\mathcal{S}'$ and exit the iteration. Note that in this case, both $\mathcal{S}$ and $\mathcal{S}'$ are not realizable.

Return $\mathcal{S}'$ as the output of **Encode**($\mathcal{S}$).

Dec: Given the set of samples $\mathcal{S}'$ as the encoding, return the version space $\mathcal{H}(\mathcal{S}')$.

Proof of version space compression. We next show that for any set $\mathcal{S}$, $\text{Dec}(\text{Enc}(\mathcal{S})) = \mathcal{H}(\mathcal{S})$. For this, we prove that $\mathcal{H}(\mathcal{S}) = \mathcal{H}(\mathcal{S}')$, using induction on number of samples that have been iterated over in the encoding protocol **Enc**. We prove that for all iterations indexed by i , $\mathcal{H}(\mathcal{S}'_i) = \mathcal{H}(\mathcal{S}_i)$. Here, $\mathcal{S}'_i$ denotes the set $\mathcal{S}'$ after i iterations and $\mathcal{S}_i$ denotes the set of first i samples in $\mathcal{S}$. The statement is trivially true for $i = 0$. To prove the induction step, we assume that $\mathcal{H}(\mathcal{S}'_i) = \mathcal{H}(\mathcal{S}_i)$ after i iterations, and show that the statement is true for the $(i+1)$ th iteration. The forward direction is straightforward, as by construction, $\mathcal{S}'_{i+1} \subseteq \mathcal{S}_{i+1}$ and thus $\mathcal{H}(\mathcal{S}_{i+1}) \subseteq \mathcal{H}(\mathcal{S}'_{i+1})$. We next prove the converse direction, that is $\mathcal{H}(\mathcal{S}'_{i+1}) \subseteq \mathcal{H}(\mathcal{S}_{i+1})$.

Suppose there exists a hypothesis $h \in \mathcal{H}(\mathcal{S}'_{i+1}) \setminus \mathcal{H}(\mathcal{S}_{i+1})$. Since $h \notin \mathcal{H}(\mathcal{S}_{i+1})$ but $h \in \mathcal{H}(\mathcal{S}_i) = \mathcal{H}(\mathcal{S}'_i)$, for the sample in the $(i+1)$ th iteration (say (x_{i+1}, y_{i+1})), $h(x_{i+1}) \neq y_{i+1}$. This implies that $(x_{i+1}, y_{i+1}) \notin \mathcal{S}'_{i+1}$. Under the **Enc** protocol, we don't add (x_{i+1}, y_{i+1}) to $\mathcal{S}'$ only when $\forall h' \in \mathcal{H}(\mathcal{S}'_i)$, $h'(x_{i+1}) = y_{i+1}$. This implies that $h \notin \mathcal{H}(\mathcal{S}'_i) \supseteq \mathcal{H}(\mathcal{S}'_{i+1})$, which is a contradiction. Hence, $\mathcal{H}(\mathcal{S}'_{i+1}) \subseteq \mathcal{H}(\mathcal{S}_{i+1})$.

Bound on $|\text{Enc}(\mathcal{S})|$. We next argue that for any input set $\mathcal{S} \in \mathcal{Z}^*$, the encoding $\mathcal{S}' = \text{Enc}(\mathcal{S})$ contains at most $(\epsilon(\mathcal{H}) + 1)$ samples. Let m denote the number of samples in $\mathcal{S}'$. Note that for all but last sample added to $\mathcal{S}'$, the first $m-1$ samples in $\mathcal{S}'$ are realizable via $\mathcal{H}$. Thus, there exists a function h' such that $h'(x_{i_j}) = y_{i_j}$ for all samples $(x_{i_j}, y_{i_j}) \in \mathcal{S}'$ where $j \leq m-1$. Next, note that, by construction of the set $\mathcal{S}'$ in the **Enc** protocol, the sample (x_{i_j}, y_{i_j}) is added to $\mathcal{S}'$ iff there exist two hypothesis $h_1, h_2 \in \mathcal{H}(\mathcal{S}'_{i_j-1}) \subseteq \mathcal{H}(\mathcal{S}'_{i_j-1})$ such that $h_1(x_{i_j}) = y_{i_j}$ and $h_2(x_{i_j}) \neq y_{i_j}$. Thus, the first $m-1$ samples in $\mathcal{S}'$ form an eluding sequence w.r.t. the hypothesis class $\mathcal{H}$. Thus, we must have that $m-1 \leq \epsilon(\mathcal{H})$. This implies that the number of compression bits C used by **Enc** satisfy $C \leq O(\epsilon(\mathcal{H}) \log(|\mathcal{Z}|))$. $\blacksquare$

D.5. Proof of Lemma 20

Proof of Lemma 20. Let a set of functions **Encode**, **Merge** and **Decode** make a C -bit mergeable compression scheme for $\mathcal{H}$. We show that there exists a function $\text{Dec} : \{0, 1\}^C \rightarrow 2^{\mathcal{H}}$ such that the pair of functions (**Encode**, **Dec**) is a C -bit version-space compression scheme for $\mathcal{H}$. Give a C -bit compression string E , **Dec** outputs a subset $\mathcal{G} \subseteq \mathcal{H}$:

1. Initialize $\mathcal{G} = \emptyset$.
2. For all $h \in \mathcal{H}$, execute the following steps
 - (a) Let $\mathcal{S}_h = \{(x, y) \mid x \in \mathcal{X} \text{ and } y = h(x)\}$.
 - (b) Let $F = \text{Encode}(\mathcal{S}_h)$.
 - (c) If $\text{Decode}(\text{Merge}(E, F)) = \text{Yes}$, then add h to $\mathcal{G}$, otherwise go to the next hypothesis.
3. Output $\mathcal{G}$.

To prove that (**Encode**, **Dec**) is a version-space compression scheme, we want to show that for all sample sets $\mathcal{S} \in (\mathcal{X} \times \mathcal{Y})^*$, $\text{Dec}(\text{Encode}(\mathcal{S})) = \mathcal{H}(\mathcal{S})$. Let $E = \text{Encode}(\mathcal{S})$ and $\mathcal{G}$ be the subset outputted when the above **Dec** algorithm is run on E . Below, we prove that

$$h \in \mathcal{G} \text{ if and only if } h \in \mathcal{H}(\mathcal{S}).$$

Recall that $\mathcal{H}(\mathcal{S}) = \{h \in \mathcal{H} \mid h(x) = y \text{ for all } (x, y) \in \mathcal{S}\}$.

Case 1 ($h \in \mathcal{H}(\mathcal{S})$). Consider the set $\mathcal{S} \cup \mathcal{S}_h$ (where $\mathcal{S}_h$ is defined in Step 2a of the **Dec** algorithm). As $h \in \mathcal{H}(\mathcal{S})$, all samples in $\mathcal{S}$ agrees with h and hence, the set $\mathcal{S} \cup \mathcal{S}_h$ is realizable. By the definition of mergeable compression $\text{Merge}(\text{Encode}(\mathcal{S}), \text{Encode}(\mathcal{S}_h)) = \text{Encode}(\mathcal{S} \cup \mathcal{S}_h)$. As $\mathcal{S} \cup \mathcal{S}_h$ is realizable, $\text{Decode}(\text{Merge}(E, \text{Encode}(\mathcal{S}_h))) = \text{Yes}$ and thus, h is added to $\mathcal{G}$.

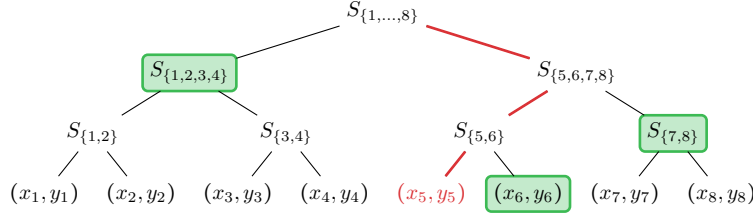


Figure 2: Picture taken from Ghazi et al. (2023). Illustration of TiLU scheme underlying the proof of Theorem 13. The ticket t_5 for the example (x_5, y_5) are the index i and the outputs of Encode applied on the subsets $S_{\{1,2,3,4\}}$, $S_{\{7,8\}}$, and $S_{\{6\}}$.

Case 2 ($h \notin \mathcal{H}(S)$). Similar to Case 1, consider the set $S \cup S_h$. We will prove that $S \cup S_h$ is not realizable. This would further imply that $\text{Decode}(\text{Merge}(E, \text{Encode}(S_h))) = \text{Decode}(\text{Encode}(S \cup S_h)) = \text{No}$ and hence h is not added to $\mathcal{G}$ at Step 2c of the Dec algorithm. As $h \notin \mathcal{H}(S)$, there exists a sample $(x, y) \in S$ such that $y \neq h(x)$. As S_h contains the sample $(x, h(x))$, both (x, y) and $(x, h(x))$ belongs to the set $S \cup S_h$ for a $y \neq h(x)$, implying that it is not realizable. ■

D.6. Proof of Theorem 13

The following proof is a straightforward extension of the proof of (Ghazi et al., 2023, Theorem 5) and is included here for completeness.

Proof. For simplicity, assume that n is a power of 2; the argument generalizes to all n easily. Let the samples in S be $\{(x_i, y_i)\}_{i \leq n}$ (arranged in any order), and consider a full binary tree of depth $d = \log_2 n$ with leaf i corresponding to the sample (x_i, y_i) . For each internal node v , let $S_v := \{(x_i, y_i) \mid v \text{ is an ancestor of the leaf } i\}$ be the dataset consisting of examples corresponding to the leaf nodes in the subtree under v . For any leaf i , let $v_1, \dots, v_{d-1}$ be the nodes on the path from the root of the tree to leaf i , and for each $j \in \{2, \dots, d\}$ let $\tilde{v}_j$ be the child of v_{j-1} and sibling of v_j . Figure 2 shows an example. Let the ticket corresponding to example i be given as

$$\text{tx}_i := (i, \text{Encode}(S_{\tilde{v}_2}), \dots, \text{Encode}(S_{\tilde{v}_d})).$$

It is immediate to see that the number of bits in tx_i is $d + C \cdot (d - 1)$, where $d = \log(n)$. Define $\text{Learn}(S)$ to return $\text{aux} = \text{Decode}(\text{Encode}(S))$ and tickets tx_i as specified above.

We define Unlearn as follows. If $I = \emptyset$, then simply return aux . Given a non-empty $I \subseteq [n]$, let R be the set of all nodes v such that no leaf in the sub-tree under v belongs to I , but the same is not true of the parent of v . It is easy to see that $S \setminus S_I$ is precisely given as $\bigcup_{v \in R} S_v$, and moreover, S_v and $S_{v'}$ are disjoint for distinct $v, v' \in R$. For all $v \in R$, we can recover $\text{Encode}(S_v)$ from ticket tx_i for any leaf $i \in I$ in the subtree under the sibling of v . Thus, by repeated applications of Merge , we can recover $\text{Encode}(\bigcup_{v \in R} S_v) = \text{Encode}(S \setminus S_I)$. Thus, we can compute the $\{\text{Yes}, \text{No}\}$ answer after unlearning using $\text{Decode}(\text{Encode}(\bigcup_{v \in R} S_v))$. ■

D.7. Missing Details from Section 4.2

In the following, we provide an implementation of the **Canonical** function, used in the proof of Lemma 17.

Canonical: Given a subset $\mathcal{H}' \subseteq \mathcal{H}$ as the input, if $\mathcal{H}' = \emptyset$, return $\{(x, 0), (x, 1)\}$ for a fixed $x \in \mathcal{X}$, otherwise initialize $S' = \emptyset$, and grow it as follows: For $x \in \text{LexicographicallySort}(\mathcal{X})$:

- Update $S' = S' \cup \{(x, y)\}$ if $h'(x) = y$ for all $h' \in \mathcal{H}'$ but there exists an $h \in \mathcal{H}$ such that $h(x) \neq y$. Else, continue to the next sample.

Check if $\mathcal{H}(S') = \mathcal{H}'$. If yes return S' , otherwise, return $\perp$.

Here, we prove a bound on the size of a version space compression scheme for parities which is relevant to the results of Remark 30.

Lemma 27 *Any C -bit version-space compression scheme for the class of parities $\mathcal{H}_{\text{parity}}$ in dimension d must satisfy:*

$$C \geq \frac{d(d-6)}{2}.$$

Furthermore, this is also true when restricted to datasets of size at most $d/2$.

Proof. Consider a dataset generated through the following process:

- First choose a subspace V of $\{0, 1\}^d$
- Choose a set of $d/2$ linearly independent vectors, $\{x_i\}_{i=1}^{d/2}$ in V
- Generate dataset $S = \{(x_i, 1)\}_{i=1}^{d/2}$.

Let $w^* \in \{0, 1\}^d$ be any solution to the dataset. Then, observe that:

$$V_{\perp} = \{w - w^* : w \in \mathcal{H}(S)\}$$

where $V_{\perp}$ denotes the orthogonal complement of the sub-space V which is also a subspace of dimension $d/2$. Hence, $C + d$ bits corresponding to **Encode**(S) with a canonical solution, w^* with the decoding function, **Decode**, suffice to recover S . However, the choice of S is arbitrary and there exist [Kac and Cheung \(2002\)](#):

$$\binom{d}{d/2}_2 := \frac{(1-2^d)(1-2^{d-1}) \dots (1-2^{d/2+1})}{(1-2) \dots (1-2^{d/2})}$$

possible choices for S . From the following observation,

$$\frac{1-q^t}{1-q^{t-d/2}} \geq 2^{d/2-1},$$

we get that:

$$\binom{d}{d/2}_2 \geq 2^{d(d-2)/4}.$$

Hence, we must have:

$$d + C \geq \frac{d(d-2)}{4} \implies C \geq \frac{d(d-6)}{2}$$

concluding the proof. ■

D.8. Version-space Compression is Necessary for Unlearning in the Central Model

[Theorem 29](#) shows that version-space compression is necessary for unlearning in the central model. Before we state and prove the theorem, we define a minimum identification set for a hypothesis class.

Definition 28 (Minimum Identification Set) *For any hypothesis class $\mathcal{H}$, minimum identification set $\mathcal{T} \subseteq \mathcal{X}$ is the minimum sized subset such that knowing the labels on this set, identifies the hypothesis h that generated them. Formally,*

$$\mathcal{T} = \operatorname{argmin} |\mathcal{S}| \text{ s.t. } \forall h_1, h_2 \in \mathcal{H}, (h_1 = h_2) \text{ or } (\exists x \in \mathcal{S}, h_1(x) \neq h_2(x)).$$

Let $m(\mathcal{H})$ denote the size of the minimum identification set.

The above definition of minimum identification set is closely related to the notion of minimal specifying set in active learning ([Hanneke, 2009, 2024](#)), with the main difference being that we do not restrict ourselves to a particular labeling function on the set $\mathcal{T}$.

Note that the entire set $\mathcal{X}$ is an identification set, but for certain hypothesis class, the size of the minimum identification set can be much smaller. One such example is the class of parity functions defined on n -bit strings, that is, $\mathcal{X} = \{0, 1\}^n$. A parity function is represented by another n -bit string a and computes the parity of the inner product of a with input x , that is, $f_a(x) = (\sum_{i=1}^n a_i x_i) \bmod 2$. Thus, the class of parity functions contains all such functions – $\mathcal{H}_{\text{parity}} = \{f_a \mid a \in \{0, 1\}^n\}$. Knowing the output of a parity function on indicator strings (that is, $(1, 0, \dots, 0), (0, 1, 0, \dots, 0), \dots$) determines the function and hence, this is an identification set. Therefore, for the class of parity functions, $m(\mathcal{H}_{\text{parity}}) \leq n \ll 2^n = |\mathcal{X}|$.

In the following theorem, we show that for classes with bounded eluder dimension and small sized identification sets, LU schemes require large storage even for small sized datasets. [Theorem 10](#) beats the bound achieved by following theorem when $m(\mathcal{H}) \approx |\mathcal{X}|$.

Theorem 29 *For any hypothesis class $\mathcal{H}$, if there exists a $(0, 0)$ – LU scheme (under the central model) with space complexity $s(n)$, then there exists a version-space compression scheme for $\mathcal{H}$, which uses $s(\epsilon(\mathcal{H}) + 2m(\mathcal{H}) + 1)$ bits of compression. Here, $\epsilon(\mathcal{H})$ and $m(\mathcal{H})$ represent the eluder dimension and the size of minimum identification set for $\mathcal{H}$, respectively².*

Remark 30 *As a concrete application, for class of parity functions, where $\epsilon(\mathcal{H}_{\text{parity}}) = m(\mathcal{H}_{\text{parity}}) = n$, $(0, 0)$ – LU schemes requires $\Omega(d^2)$ bits of storage even for datasets of sample size $\Theta(d)$. This is because, version-space compression for the class of parity functions requires $\Omega(d^2)$ bits. We give proof of the last fact in [Lemma 27](#).*

Proof of Theorem 29. Let $(\text{Learn}, \text{Unlearn})$ be an LU scheme for the hypothesis class $\mathcal{H}$ with space complexity $s(n)$. We construct the pair of functions (Enc, Dec) for version-space compression for the class $\mathcal{H}$ as follows:

Enc: Given a set of samples $\mathcal{S}$, the encoding algorithm constructs an eluder subsequence $\mathcal{S}'$ as in the proof of [Theorem 18](#), which is a subset of samples in $\mathcal{S}$, such that $|\mathcal{S}'| \leq \epsilon(\mathcal{H}) + 1$ and

2. Note that, both these quantities are incomparable to each other. We give examples in [Proposition 42](#).

$\mathcal{H}(\mathcal{S}) = \mathcal{H}(\mathcal{S}')$. Let $\mathcal{T}$ be the minimum identification set for the hypothesis class $\mathcal{H}$. Let D be a dataset that contains $\mathcal{S}'$ plus two samples per $x \in \mathcal{T}$, that $(x, 0)$ and $(x, 1)$. Thus, D contains at most $\epsilon(\mathcal{H}) + 1 + 2m(\mathcal{H})$ samples. **Enc** outputs the auxiliary information that LU scheme uses for this dataset – **Learn**(D). Using the space complexity of the LU scheme, we can bound the compression bits used by the encoding algorithm by $s(|D|)$.

Dec: Given the auxiliary information as an encoding E , the decoding algorithm constructs a version-space $\mathcal{H}'$ as follows:

- Initially $\mathcal{H}' = \emptyset$.
- For all $h \in \mathcal{H}$, we construct an unlearning request:
 - Let $\mathcal{S}_h = \{(x, 1 - h(x)) \mid x \in \mathcal{T}\}$.
 - If **Unlearn**($\mathcal{S}_h, E$) answers **Yes**, then add h to $\mathcal{H}'$.
- Output $\mathcal{H}'$.

To prove the (**Enc**, **Dec**) is a version-space compression for $\mathcal{H}$, we need to show that for all sample sets $\mathcal{S}$, $\text{Dec}(\text{Enc}(\mathcal{S})) = \mathcal{H}(\mathcal{S})$. It suffices to show that $\text{Dec}(\text{Enc}(\mathcal{S})) = \mathcal{H}(\mathcal{S}')$ where $\mathcal{S}'$ is the eluder subsequence of $\mathcal{S}$. Firstly, we note that the unlearning request for each hypothesis is valid because the dataset D used for encoding contains both $(x, 0)$ and $(x, 1)$ for all $x \in \mathcal{T}$. After deletion, the updated dataset $D_h = D \setminus \mathcal{S}_h = \mathcal{S}' \cup \{(x, h(x)) \mid x \in \mathcal{T}\}$. By the learning-unlearning guarantee of the pair of functions (**Learn**, **Unlearn**), **Unlearn**($\mathcal{S}_h, E$) is **Yes** if and only if D_h is realizable. So, it suffices to show that D_h is realizable if and only if $h \in \mathcal{H}(\mathcal{S}')$.

The backward direction is straightforward – if $h \in \mathcal{H}(\mathcal{S}')$, then h realizes all the samples in D_h , and hence the dataset is realizable. To prove the other direction, we use the fact that $\mathcal{T}$ is an identification set. If D_h is realizable, then there exists $h' \in \mathcal{H}$ such that $\forall (x, y) \in D_h, h'(x) = y$. As $\forall x \in \mathcal{T}, (x, h(x)) \in D_h$, this implies that $h(x) = h'(x), \forall x \in \mathcal{T}$. By [Definition 28](#), as $\mathcal{T}$ is an identification set, $h = h'$. As D_h contains all samples in $\mathcal{S}'$, h realizes these samples too. Hence, $h \in \mathcal{H}(\mathcal{S}')$. ■

Appendix E. Missing Details from [Section 5](#)

Proof of [Lemma 22](#). The proof of the lemma will follow a greedy construction. We start by defining $S_0 := S$ and construct S_i for $i > 0$ as follows:

1. If there exists $(x, y) \in S_{i-1}$ such that $S_{i-1} \setminus \{(x, y)\}$ is unrealizable, construct $S_i := S_{i-1} \setminus \{(x, y)\}$.
2. Otherwise, stop and let $S' := S_{i-1}$

First, observe that the process must output an unrealizable set by definition as the process starts with an unrealizable set at step 0, and S_i is unrealizable if S_{i-1} is unrealizable. Now, to bound the size of S' , note that, by the termination condition, the removal of any point from S' renders it realizable. Hence, S' is a hollow star set and its size is bounded above by s_0 . ■

Proof of Theorem 24. To prove the theorem, we start by bounding the number of l -critical sets for $l \in [k]$. First, fix $l \in [k]$. We will maintain a list of prefixes of l -critical sets; that is, these are subsets which may be expanded to construct l -critical sets. To start, let $C_0 = \{()\}$ be the set of 0-length prefixes of l -critical sets. For $0 < i \leq l$, we construct C_i from C_{i-1} as follows:

- Let $P = ((x_1, y_1), \dots, (x_{i-1}, y_{i-1})) \in C_{i-1}$ be a prefix of an l -critical set.
- Since, P is a prefix of an l -critical set, $S \setminus P$ is unrealizable.
- From Lemma 22, there exists a subset $S' \subset S \setminus P$ which is also unrealizable and has $|S'| \leq \mathfrak{s}_o$.
- Any completion of P to a l -critical set must feature one of the elements of S'
- For all $(x, y) \in S'$ such that $P \cup \{(x, y)\}$ is extendible to an l -critical set, add $P \cup \{(x, y)\}$ to C_i

It is clear from the above construction that $|C_i| \leq \mathfrak{s}_o |C_{i-1}|$. Furthermore, note that any l -critical set $((x_1, y_1), \dots, (x_l, y_l))$ is a member of C_l by applying induction on the observation that if $((x_1, y_1), \dots, (x_i, y_i)) \in C_i$, then $((x_1, y_1), \dots, (x_{i+1}, y_{i+1})) \in C_{i+1}$. Therefore, we obtain:

$$|C_l| \leq \mathfrak{s}_o^l.$$

To conclude the proof of the theorem, observe that for any dataset $S = \{(x_i, y_i)\}_{i=1}^n$, it suffices to store the set of critical sets of sizes up to k . The answer to any query may be produced simply by checking whether the points in the query contain a critical set. This produces a valid answer to the unlearning query as any set of deletions that render a dataset realizable must contain a critical set of size at most k by noting that points may be removed from the deletion query until it reduces to a critical set. The number of critical sets of size less than k is now given by:

$$\sum_{i=1}^k \mathfrak{s}_o^i \leq \mathfrak{s}_o^{k+1}.$$

This concludes the proof of the theorem. ■

Appendix F. Learning-Unlearning Schemes for Halfspaces

Next, we present a set of results for *linear* classifiers which mirror the results of Section 5. The main contribution of this section is a lower bound which shows that the bound of Theorem 24 is *tight* for linear classifiers (Theorem 34). Hence, the exponential dependence on k is *necessary* for the relatively simple setting of linear classification.

Before we proceed, note that the domain size for linear separators is uncountably infinite. To obtain finite bounds, we restrict the domain to a finite set, say the points on a grid. This incurs an additional overhead of $\log(|\mathcal{X}|)$ in memory, which scales as $\approx d$ for natural choices of such a discretization. Our choice for the lower bound is specified subsequently. First, we recall an upper bound on the hollow star number of linear separators Bousquet et al. (2020) which utilizes a result from Braverman et al. (2019). We present the proof here for convenience.

Lemma 31 (Bousquet et al. (2020)) *For the hypothesis class of linear separators, $\mathcal{H}$, we have:*

$$\mathfrak{s}_o(\mathcal{H}) \leq d + 2.$$

Proof. Let $S = \{(x_1, y_1), \dots, (x_l, y_l)\}$ be a hollow star set and $S = S_p \cup S_n$ be a decomposition of S into its positively and negatively labeled points. Since S is not realizable by any linear classifier, there exists a point $z \in \text{conv}(S_p) \cap \text{conv}(S_n)$. For this point z . Suppose $z = \sum_{x \in S_p} w_x x = \sum_{y \in S_n} w'_y y$ such that $w_x \geq 0$ for all $x \in S$ and $\sum_{x \in S_p} w_x = \sum_{y \in S_n} w_y = 1$. Let $Q = Q_p \cup Q_n$ with Q_p and Q_n denoting the supports of w and w' respectively; i.e $Q_p = \text{supp}(w)$ and $Q_n = \text{supp}(w')$. We will show that there exists a $Q \subset S$ such that $|Q| \leq d + 2$.

Claim 32 *We have:*

$$|Q| \leq d + 2.$$

Proof. Suppose for the sake of contradiction that $|Q| > d + 2$. Then, we have that there exists $\tilde{w}, \tilde{w}' \neq 0$ such that the following hold:

$$\begin{aligned} \sum_{x \in Q_p} \tilde{w}_x x &= \sum_{y \in Q_n} \tilde{w}'_y y \\ \sum_{x \in Q_p} \tilde{w}_x &= 0 \text{ and } \sum_{y \in Q_n} \tilde{w}_y = 0. \end{aligned}$$

Consider the vector, $(w_\gamma, w'_\gamma) = (w + \gamma \tilde{w}, w' + \gamma \tilde{w}')$ and the largest γ such that all the entries of w_γ and w'_γ are non-negative and the corresponding γ^* . Observe that one of the $w_{\gamma^*}, w'_{\gamma^*}$ must have a zero-coordinate. Furthermore, the vectors obtained using the weights $w_{\gamma^*}, w'_{\gamma^*}$ also lie in the intersection of the convex hulls of S_p and S_n . The size of the support of $w_{\gamma^*}, w'_{\gamma^*}$ is $|Q| - 1$. Iterating this process, we arrive at a support set of size $d + 2$. $\blacksquare$

Since S is a hollow star set, $S \setminus \{(x, y)\}$ is realizable for any $(x, y) \in S$. However, any $(x, y) \in S$ must also belong to Q as Q itself is unrealizable. 32 concludes the proof. $\blacksquare$

With Lemma 31, we obtain the following corollary of Theorem 24.

Corollary 33 *There exists a central $(0, 0)$ -LU scheme with $O((d+2)^{k+1} \log(|\mathcal{X}|))$ central memory size.*

Next, we present the main result of the section which shows that the memory size guarantees of 33 are essentially *optimal*. Furthermore, our results for a domain of size 2^d (hence, $\log(|\mathcal{X}|) = d$) defined below with e_i denoting the standard basis vectors:

$$\mathcal{X} := \bigcup_{i \in [d]} \mathcal{X}_i \text{ where } \mathcal{X}_i := \left\{ \sum_{j=1}^d \alpha_j e_j : \alpha_i \in \left\{ 0, \frac{1}{i} \right\} \text{ and } \sum_{j=1}^d \alpha_j = 1 \right\}. \quad (6)$$

Our lower bound on the space complexity of an LU scheme for linear classifiers is presented below. As remarked previously, the theorem establishes the optimality of 33 even for the ubiquitous setting of *linear* classification.

Theorem 34 *Let $d \geq 1$ and consider any $2 \leq k \leq d - 1$. For any (ϵ, δ) -LU scheme for halfspaces over the domain $\mathcal{X}$ defined in (6) of size $d + \binom{n}{k}$, there exists a worst-case dataset of size at most $n = |\mathcal{X}|$ such that the space complexity of the scheme satisfies,*

$$s(n) \geq (1 - H(\delta)) \binom{d}{k} - 1. \quad (7)$$

Notably, this lower bound applies even when the scheme must only answer unlearning queries of size at most k .

Proof. We start by constructing a family of datasets and show that the central LU scheme *must* use a large amount of memory for at least one member of the family. Our family of datasets will be indexed by subsets of the set $\mathcal{I}$ which consists of all size- k tuples of elements from $[d]$; i.e:

$$\mathcal{I} := \{\{i_1, \dots, i_k\} : \forall j \in [k], i_j \in [d], \forall l \neq j : i_j \neq i_l\}.$$

Now, for any subset $S \subseteq \mathcal{I}$, our dataset, denoted by D_S , is defined as follows:

1. Add $(e_i, 1)$ to D_S for all $i \in [d]$
2. For all $L = \{i_1, \dots, i_k\} \in S$, add:

$$\left(x = \frac{1}{d-k} \sum_{j \notin L} e_j, y = 0 \right)$$

to the dataset. That is, x corresponds to the mean of the standard basis vectors corresponding to the elements not in L , and the corresponding label is 0.

Hence, the dataset is of size $|S| + d$. We now establish the following structural property on the critical sets of D_S that enables identifying S from the set of critical sets. Denoting by Q_S the set of size- k critical sets of D_S , we show the following.

Claim 35 *We have:*

$$\begin{aligned} \forall \{i_1, \dots, i_k\} \in S : \{(e_{i_1}, 1), \dots, (e_{i_k}, 1)\} &\notin Q_S \\ \forall \{i_1, \dots, i_k\} \notin S : \{(e_{i_1}, 1), \dots, (e_{i_k}, 1)\} &\in Q_S \end{aligned}$$

Proof. Let $L = \{i_1, \dots, i_k\}$. We will prove each of the sub-claims in turn. For the first, assume the contrary that there exists a linear classifier (w, b) which correctly classifies the dataset after the removal of $e_{i_1}, \dots, e_{i_k}$. Therefore, we simultaneously have:

$$\forall i \notin L : \langle e_i, w \rangle \geq b$$

for the positive labels and

$$\left\langle \frac{1}{d-k} \sum_{i \notin L} e_i, w \right\rangle < b$$

for a one of the negative data points in the dataset. This is a contradiction which establishes the first sub-claim. For the second sub-claim, consider the linear classifier defined by w, b :

$$w := \sum_{i \notin L} e_i \text{ and } b = 1 - \frac{1}{2(d-k)}.$$

We have for any $i \notin L$:

$$\langle w, e_i \rangle = 1 > b.$$

Now, we have for any $L' = \{i'_1, \dots, i'_k\} \neq L$:

$$\left\langle w, \frac{1}{d-k} \sum_{i \notin L'} e_i \right\rangle = 1 - \frac{|L' \setminus L|}{d-k} < b.$$

This proves the second sub-claim. ■

Let by S be a uniformly random subset of $\mathcal{I}$ (represented in binary). Let D_S denote the dataset corresponding to S . Note that 35, if the answer of the unlearning query was exactly correct then

$$\{L = \{i_1, \dots, i_k\} \in \mathcal{I} : \text{Unlearn}(\{e_{i_1}, \dots, e_{i_k}\}, p_S) = \text{No}\} = S.$$

Accounting for errors, we have that

$$\Pr[L \in S \mid \text{Unlearn}(L, \text{Learn}(D_S)) = \text{No}] \geq 1 - \delta$$

This implies

$$\mathbb{H}(Z_L \mid \text{Unlearn}(L, \text{Learn}(D_S))) \leq 1 - \mathbb{H}(\delta)$$

As in the proof of Theorem 7, we have that the

$$|\text{Learn}(D)| \geq (1 - \mathbb{H}(\delta))|\mathcal{I}|$$

The lower bound follows noticing that $|\mathcal{I}| = \binom{d}{k}$. ■

As a corollary of the proof, we also obtain the following corollary for the *ticketed* setting by observing that only the tickets of the *positively* labelled points (that is, the tickets of $\{e_i\}_{i \in [d]}$), suffice for reconstructing the precise subset in $\mathcal{I}$.

Corollary 36 *$d \geq 1$ and $2 \leq k \leq d-1$. Consider any (ϵ, δ) -TiLU scheme which supports unlearning queries of size at most k , for halfspaces over the domain defined in (6) of size $d + \binom{n}{k}$. Then, there exists a worst-case dataset of size at most $n = |\mathcal{X}|$ such that the space complexity of the scheme satisfies,*

$$s(n) = \Omega\left(\frac{1 - \mathbb{H}(\delta)}{d} \cdot \binom{d}{k}\right). \quad (8)$$

As remarked previously, the corollary follows from the observation that only the tickets of the points $(e_i, 1)$ are required for the reconstruction procedure defined in the proof of Theorem 34.

We finally note that the margin of the linear classifiers considered in the proof of Theorem 34 scale polynomially in d , and it appears likely that restriction to *constant* margin is required to obtain improved bounds on the space complexity of LU schemes and their ticketed variants.

Remark 37 *The margin of the linear classifiers utilized in the proof of Theorem 34 is $1/(2d)$.*

Appendix G. Relation Between Various Combinatorial Dimensions

Definition 38 (Eluder dimension (Russo and Van Roy, 2013)) *The eluder dimension of a hypothesis class $\mathcal{H}$ over a domain $\mathcal{X}$ is defined as the largest value of ℓ such that there exists an eluder sequence of length ℓ . $(x_1, y_1), \dots, (x_\ell, y_\ell)$ is said to be an eluder sequence if for each $i \leq \ell - 1$, the version space of $(x_1, y_1), \dots, (x_i, y_i)$ contains hypotheses h, h' such that $h(x_{i+1}) \neq h'(x_{i+1})$.*

Bounds on the eluder dimension for various function classes are well known, e.g. when $\mathcal{F}$ is finite, $\epsilon(\mathcal{H}) \leq |\mathcal{H}| - 1$, and when $\mathcal{H}$ is the set of d -dimensional function with bounded norm, then $\epsilon(\mathcal{H}) = O(d)$. We refer the reader to Russo and Van Roy (2013); Mou et al. (2020); Li et al. (2022) for more examples.

Definition 39 (Littlestone dimension) *Given a hypothesis class $\mathcal{H}$ that maps elements from a domain $\mathcal{X}$ to $\{0, 1\}$, the littlestone tree is a rooted binary tree, where each node is labeled by a point $x \in \mathcal{X}$, the left edge is labeled by $y = 0$, and the right edge is labeled by $y = 1$. The tree is said to be shattered by a hypothesis class $\mathcal{H}$ if, for every branch in the tree there exists a concept $h \in \mathcal{H}$ which is consistent with the edges of the branch (i.e. for every x and the decision y on this branch, we have $h(x) = y$).*

The Littlestone dimension of the hypothesis class $\mathcal{H}$, denoted by $l(\mathcal{H})$, is the largest $l \in \mathbb{N} \cap \{0\}$ for which there exists a complete littlestone tree of depth l all of whose branches are shattered by $\mathcal{H}$.

When clear from the context, we use the notation l to denote the $l(\mathcal{H})$.

Lemma 40 (Li et al. (2022); Hanneke (2024)) *For any hypothesis class $\mathcal{H}$,*

- $d(\mathcal{H}) \leq \mathfrak{s}(\mathcal{H}) \leq \epsilon(\mathcal{H}) \leq |\mathcal{H}| - 1$,
- $l(\mathcal{H}) \leq \epsilon(\mathcal{H})$,
- $\log(|\mathcal{H}|) \leq \epsilon(\mathcal{H})$,

where $d(\mathcal{H})$, $\mathfrak{s}(\mathcal{H})$, $l(\mathcal{H})$, and $\epsilon(\mathcal{H})$ denote the VC-dimension, star number, Littlestone dimension and eluder dimension of $\mathcal{H}$, respectively.

The next result relates the hollow-star number for a hypothesis class, to its star number.

Lemma 41 *For any hypothesis class $\mathcal{H}$, the star number is always larger than the hollow star number, i.e.*

$$\mathfrak{s}_o(\mathcal{H}) - 1 \leq \mathfrak{s}(\mathcal{H}).$$

Proof. We will show that for any hollow star set of size l , there exists a star set of size $l - 1$. Let $S = \{(x_1, y_1), \dots, (x_l, y_l)\}$ be a hollow star set. By definition, S is not realizable and for any $i \in [l]$:

$$S_i = \{(x_j, y_j)\}_{j \neq i} \cup \{(x_i, \bar{y}_i)\}$$

is realizable. Consider the set $S^* = \{(x_1, y_1), \dots, (x_{l-1}, y_{l-1})\}$. From the fact that S_n is realizable, S^* is realizable. Furthermore, from the fact that S_i is realizable for $i \neq n$, S^* is also a star set. Hence, there exists a star set of size $l - 1$. This concludes the proof. $\blacksquare$

The following technical result shows that the minimum identification set and the eluder dimension are incomparable.

Proposition 42 For any $n \in \mathbb{N}$,

- There exists a domain, $\mathcal{X}_1$, and hypothesis set $\mathcal{H}_1$ such that $|\mathcal{X}_1| = n, |\mathcal{H}_1| = 2n$ such that:

$$\mathfrak{m}(\mathcal{H}_1) = n \text{ and } \mathfrak{e}(\mathcal{H}_1) \leq 32 \log(n).$$

- There exists a domain, $\mathcal{X}_2$, and hypothesis set $\mathcal{H}_2$ such that $|\mathcal{X}_2| = n + \log(n), |\mathcal{H}_2| = n$ such that:

$$\mathfrak{m}(\mathcal{H}_2) = \log(n) \text{ and } \mathfrak{e}(\mathcal{H}_2) \geq n - 1.$$

Proof. We first establish the sub-claim that the minimum identification set may be substantially larger than eluder dimension. We will use a randomized construction to show the existence of such a set. Our domain will correspond to the set of integers up to n ; i.e $\mathcal{X} := [n]$, and the hypothesis set will consist of $2n$ functions indexed by $[n] \times \{0, 1\}$, $\mathcal{H} := \{h_{i,l}\}_{i \in [n], l \in \{0,1\}}$. For each $i \in [n]$, $h_{i,0}, h_{i,1}$ are constructed as follows:

$$\begin{aligned} h_{i,0}(i) &= 0 \text{ and } h_{i,1}(i) = 1 \\ \forall j \neq i : h_{i,0}(j) &= h_{i,1}(j) = W_{i,j} \text{ where } W_{i,j} \stackrel{\text{iid}}{\sim} \text{Unif}(\{0, 1\}). \end{aligned}$$

We will show that with non-zero probability, the constructed hypothesis set has eluder dimension bounded above by $O(\log(n))$ and minimum identification set bounded below by $\Omega(n)$. We start with the simpler lower bound on $\mathfrak{m}$.

Claim 43 We have:

$$\mathfrak{m} = n.$$

Proof. Suppose $S \subset [n]$ with $|S| \neq n$ be a minimum identification set. This is a contradiction as for any $i \notin S$, $h_{i,0}$ and $h_{i,1}$ realize the same values on the elements in $\mathcal{X}$. $\blacksquare$

We next show that the eluder dimension is small. Here, we assume that in addition to the random variables $W_{i,j}$ drawn in the construction of $\mathcal{H}$, we also assume for the sake of convenience that $W_{i,i}$ is drawn. We prove the following simple

Claim 44 We have:

$$\Pr \left\{ \exists S_{\mathcal{X}}, S_{\mathcal{H}} : |S_{\mathcal{X}}|, |S_{\mathcal{H}}| \geq 4 \log(n) \text{ and } \forall i \in S_{\mathcal{X}}, \forall j, j' \in S_{\mathcal{H}}, W_{i,j} = W_{i,j'} \right\} < 1.$$

Proof. Let $l = 4 \log(n)$. We will show this via a union bound over all possible subsets $S_{\mathcal{X}}, S_{\mathcal{H}}$. First, fix two sets $S_{\mathcal{X}}$ and $S_{\mathcal{H}}$ of size l . Now, we have that

$$\Pr \left\{ W_{i,j} = W_{i,j'} \text{ for all } i \in S_{\mathcal{X}}, j, j' \in S_{\mathcal{H}} \right\} = 2^{-l(l-1)}.$$

Hence, we get by a union bound over all such possible $S_{\mathcal{X}}, S_{\mathcal{H}}$ that the probability that there exists such a pair of sets is upper bounded by:

$$\binom{n}{l} \cdot \binom{n}{l} \cdot 2^{-l(l-1)} \leq n^{2l} \cdot 2^{-l(l-1)} < 1$$

for our value of l . $\blacksquare$

We will condition on the events of 44. Suppose for the sake of contradiction that there existed an eluder sequence over the sequence of elements $\{i_1, \dots, i_l\}$ in $[n]$ with $l = 32 \log(n)$. Let the base function of the sequence be h_0 and $h_1, \dots, h_l$ be the deviations from h_0 with $h_j(i_j) \neq h_0(i_j)$ and $h_j(i_k) = h_0(i_k)$ for all $k < j$. Now, observe the following:

$$\forall j > 4 \log(n), k \leq 4 \log(n) : h_j(i_k) = h_0(i_k).$$

Now, define the sets $\mathcal{S}'_{\mathcal{X}}, \mathcal{S}'_{\mathcal{H}}$ as follows:

$$\mathcal{S}'_{\mathcal{H}} = \{h_j\}_{j > 4 \log(n)} \setminus \{h_{i_k, y}\}_{k \in [4 \log(n)], y \in \{0, 1\}} \text{ and } \mathcal{S}_{\mathcal{X}} = \{i_k\}_{k \leq 4 \log(n)}.$$

That is, $\mathcal{S}'_{\mathcal{H}}$ consists of the hypotheses whose values are *entirely* determined by the random variables $W_{i, j}$ on the indices in $\mathcal{S}_{\mathcal{X}}$. We further prune $\mathcal{S}'_{\mathcal{H}}$ to remove duplicates as follows:

$$\mathcal{S}_{\mathcal{H}} = \mathcal{S}'_{\mathcal{H}} \setminus \{h_{i, 1} : h_{i, 0} \in \mathcal{S}'_{\mathcal{H}}\}.$$

Since this process removes at most one hypothesis of a pair already existing in $\mathcal{S}'_{\mathcal{H}}$, we have:

$$|\mathcal{S}_{\mathcal{H}}| \geq \frac{|\mathcal{S}'_{\mathcal{H}}|}{2} \geq 10 \log(n).$$

$$h \in \mathcal{S}_{\mathcal{H}}, i \in \mathcal{S}_{\mathcal{X}} : h(i) = W_{j_h, i} \text{ for some } j_h.$$

Observing that the index j_h is *unique* for each element in $\mathcal{S}_{\mathcal{H}}$ as the duplicates are removed, we have obtained $\mathcal{S}_{\mathcal{X}}, \mathcal{S}_{\mathcal{H}}$ with size larger than $4 \log(n)$ such that for all $i \in \mathcal{S}_{\mathcal{X}}, j, j' \in \mathcal{S}_{cH}$:

$$W_{i, j} = W_{i, j'}$$

yielding a contradiction with 44.

We now show that the eluder dimension may be much larger than the minimum identification set. We will define a hypothesis set of size n over a domain of size $n + \log(n)$. Denoting the domain as $\mathcal{X} = \{a_i\}_{i \in [n]} \cup \{b_j\}_{j \in \log(n)}$, the hypotheses $\mathcal{H} = \{h_i\}_{i \in [n]}$ are defined as follows:

$$h_i(a_j) = \begin{cases} 1 & \text{if } j = i \\ 0 & \text{otherwise} \end{cases} \text{ and } h_i(b_j) = \text{Parity} \left(\left\lfloor \frac{i}{2^j} \right\rfloor \right)$$

where Parity denotes the parity of the input (0 if the input is even and odd otherwise). That is, on the inputs b_j , h_i evaluates to the elements of binary representation of i . Hence, the set $\{b_j\}_{j \in [\log(n)]}$ is a minimum identification set for $\mathcal{H}$. The minimality follows from the fact that the minimum identification set must be of size at least $\log(n)$. Finally, observe that $\mathcal{H}$ has an eluder sequence of size $n - 1$ on the elements $\{a_1, \dots, a_{n-1}\}$ with h_n as the base function and $h_1, \dots, h_{n-1}$ being the deviations from the base function on the sequence. $\blacksquare$

Appendix H. From Realizability Testing to Unlearning

In this section, we discuss the connection of our results on LU/ TiLU schemes for realizability testing, to designing unlearning schemes for empirical risk minimization under unbounded deletions. We focus on 0, 0-LU and TiLU schemes in this section, but the lower bounds hold for any constant $\varepsilon \in [0, 1]$ and $\delta \in [0, 1/2]$.

Definition 45 (Empirical risk minimization) Let $\mathcal{H}$ be a learning task. The learning task of empirical risk minimization operates on the range space $\mathcal{W} = \mathcal{H}$ and is defined as,

$$f(D) = \operatorname{argmin}_{h \in \mathcal{H}} \sum_{(x,y) \in D} \mathbb{I}(h(x) \neq y). \quad (9)$$

Ties in the $\operatorname{argmin}(\cdot)$ may be resolved arbitrarily.

The space complexity of LU schemes for ERM are denoted by $s_{\text{ERM}}(\cdot)$ to make the distinction with the notation for realizability testing clear.

In the case where the underlying dataset D is $\mathcal{H}$ -realizable, the existence of LU/ TiLU schemes for realizability testing is trivial since any subset of D is always going to remain $\mathcal{H}$ -realizable. However, under the same constraint, the existence of LU/ TiLU schemes for empirical risk minimization is non-trivial. Indeed, the challenge arises from the fact that correctness of the scheme requires that $\text{Unlearn}(D_I, \text{aux})$ (with aux computed on the input dataset D) is required to match with the hypothesis that would have been returned by $\text{Learn}(D \setminus D_I)$. Since the unlearning query D_I is arbitrary, this is a non-trivial requirement - the unlearning scheme has to output the same hypothesis whether (i) the dataset D was learned and D_I was subsequently unlearned, or (ii) whether the smaller dataset $D \setminus D_I$ was directly learned. The key technical challenge in proving the correctness of any such unlearning scheme is to exhibit that the hypothesis output after unlearning a subset of points is “canonical” in a sense.

In the case where D is $\mathcal{H}$ -realizable to begin with, we show that it is possible to devise a ticketed unlearning scheme for ERM with space complexity scaling with the eluder dimension of $\mathcal{H}$ and logarithmically in the size of D . We follow a similar approach as the case of realizability testing, and introduce the notion of a mergeable concept class for ERM to build up to the main result.

Definition 46 (Mergeable Concept Class for ERM, (Ghazi et al., 2023)) A concept class $\mathcal{H} \subseteq \{\mathcal{X} \rightarrow \mathcal{Y}\}$ is said to be C -bit mergeable if there exist methods $\text{Encode}, \text{Merge}, \text{Decode}$ such that

- $\text{Encode} : \mathcal{Z}^* \rightarrow \{0, 1\}^C$ is a permutation-invariant encoding of its input into C bits.
- $\text{Decode} : \{0, 1\}^C \rightarrow \mathcal{H}$ such that $\text{Decode}(\text{Encode}(S)) \in \text{ERM}_{\mathcal{H}}(S)$ for all $\mathcal{H}$ -realizable $S \in \mathcal{Z}^*$.
- $\text{Merge} : \{0, 1\}^C \times \{0, 1\}^C \rightarrow \{0, 1\}^C$ such that for all $S_1, S_2 \in \mathcal{Z}^*$ such that $S_1 \cup S_2^3$ is $\mathcal{H}$ -realizable, it holds that $\text{Encode}(S_1 \cup S_2) = \text{Merge}(\text{Encode}(S_1), \text{Encode}(S_2))$.

The following result by Ghazi et al. (2023) show that any C -bit mergeable hypothesis class for ERM has a TiLU scheme with bounded memory for realizable datasets.

Theorem 47 (Ghazi et al. (2023)) For any C -bit mergeable concept class $\mathcal{H}$, there exists a TiLU scheme for ERM with both the ticket size $O(C \log n)$ and auxiliary number of bits $\log |\mathcal{H}|$.

Theorem 48 (Realizable unlearning upper bound under bounded eluder dimension) let $\mathcal{H}$ be a hypothesis class with eluder dimension $\epsilon(\mathcal{H})$. Then, $\mathcal{H}$ has a TiLU scheme for ERM with both ticket size bounded by $O(C \log(n))$ and the number of auxiliary bits bounded by $\log(\mathcal{H})$.

3. We use $S_1 \cup S_2$ to denote the concatenation of the two datasets.

Proof. We first prove that the hypothesis class $\mathcal{H}$ is $O(\epsilon(\mathcal{H}) \log(|\mathcal{Z}|))$ -bit Mergeable. We first define additional notation. Recall that a set of hypothesis $\mathcal{H}' \subseteq \mathcal{H}$ is said to be a version space if there exists some dataset $\mathcal{S} \in \mathcal{Z}^*$ such that $\mathcal{H}' = \mathcal{H}(\mathcal{S})$. Next, define a function $\text{Canonical} : 2^{\mathcal{H}} \mapsto \mathcal{Z}^*$ that maps subsets $\mathcal{H}' \subseteq \mathcal{H}$ to datasets $\mathcal{S}'$ as follows: On input $\mathcal{H}'$,

- Initialize $\mathcal{S}' = \emptyset$, and for $x \in \text{LexicographicallySort}(\mathcal{X})$:
 - Update $\mathcal{S}' = \mathcal{S}' \cup \{(x, y)\}$ if $h'(x) = y$ for all $h' \in \mathcal{H}'$ but there exists an $h \in \mathcal{H}$ such that $h(x) \neq y$. Else, continue to the next sample.
- Return $\text{Canonical}(\mathcal{H}') = \mathcal{S}'$.

Claim 49 *Let $\mathcal{S}' = \text{Canonical}(\mathcal{H}(\mathcal{S}))$, we have $\mathcal{H}(\mathcal{S}) = \mathcal{H}(\mathcal{S}')$.*

Proof. The proof follows trivially from the definition of Canonical . ■

Claim 50 $|\mathcal{S}'| \leq \epsilon(\mathcal{H})$

Proof. The sequence of points $\mathcal{S}'$ form an eluding sequence w.r.t. $\mathcal{H}$ and thus we must have that $|\mathcal{S}'| \leq \epsilon(\mathcal{H})$. ■

Finally, we define the ERM rule such that for any dataset $\mathcal{S}$, $h = \text{ERM}(\mathcal{S})$ denotes the lexicographically smallest hypothesis in the version space $\mathcal{H}(\mathcal{S})$.

We are now ready to provide **Encode**, **Decode** and **Merge** methods:

Encode: Given an input $\mathcal{S}$ (that is realizable via the hypothesis class $\mathcal{H}$), compute the version space $\mathcal{H}' = \{h \in \mathcal{H} \mid h(x) = y \text{ for all } (x, y) \in \mathcal{S}\}$, and let $\mathcal{S}' = \text{Canonical}(\mathcal{H}')$. Return $\text{Encode}(\mathcal{S}) = \mathcal{S}'$.

Decode: Given an encoding $\mathcal{S}'$, compute the version space $\mathcal{H}' \leftarrow \mathcal{H}(\mathcal{S}')$, and return the lexicographically smallest hypothesis h' in $\mathcal{H}'$.

Merge: Given encodings $\mathcal{S}'_1$ and $\mathcal{S}'_2$, compute the version spaces $\mathcal{H}_1 \leftarrow \mathcal{H}(\mathcal{S}'_1)$ and $\mathcal{H}_2 \leftarrow \mathcal{H}(\mathcal{S}'_2)$, and define $\mathcal{H}'' = \mathcal{H}_1 \cap \mathcal{H}_2$. Compute the dataset $\mathcal{S}'' = \text{Canonical}(\mathcal{H}'')$ and return $\text{Merge}(\mathcal{S}'_1, \mathcal{S}'_2) = \mathcal{S}''$.

$\mathcal{H}$ is $O(\epsilon(\mathcal{H})) \log(|\mathcal{Z}|)$ -bit mergeable for ERM. Let $\mathcal{S}'$ be the output of **Encode** for a dataset $\mathcal{S}$. By definition of Canonical , we have that $\mathcal{H}(\mathcal{S}') = \mathcal{H}' = \mathcal{H}(\mathcal{S})$. Thus, $\text{Decode}(\text{Encode}(\mathcal{S})) = \text{ERM}(\mathcal{S})$.

We next establish the mergeability property. Note that for any sets $\mathcal{S}_1$ and $\mathcal{S}_2$, the output of $\text{Encode}(\mathcal{S}_1 \cup \mathcal{S}_2) = \mathcal{S}'$ where $\mathcal{S}' = \text{Canonical}(\mathcal{H}(\mathcal{S}_1 \cup \mathcal{S}_2))$. On the other hand, we compute $\text{Merge}(\text{Encode}(\mathcal{S}_1), \text{Encode}(\mathcal{S}_2)) = \mathcal{S}''$ where $\mathcal{S}'' = \text{Canonical}(\mathcal{H}(\mathcal{S}_1) \cap \mathcal{H}(\mathcal{S}_2))$. However, since $\mathcal{H}(\mathcal{S}_1) \cap \mathcal{H}(\mathcal{S}_2) = \mathcal{H}(\mathcal{S}_1 \cup \mathcal{S}_2)$ by definition of the version spaces, we have that $\mathcal{S}' = \mathcal{S}''$ and thus $\text{Merge}(\text{Encode}(\mathcal{S}_1), \text{Encode}(\mathcal{S}_2)) = \text{Encode}(\mathcal{S}_1 \cup \mathcal{S}_2)$.

Since all generated encodings have at most $\epsilon(\mathcal{H})$ -samples (due to Claim 50), we immediately get that $\mathcal{H}$ is $\epsilon(\mathcal{H}) \log(|\mathcal{Z}|)$ -bit mergeable. ■

So far, we have considered the realizable setting for unlearning for ERM. Unfortunately, there are simple examples where if the initial dataset is not realizable, then there cannot exist any LU scheme with sublinear memory.

Remark 51 (Ghazi et al. (2023, Theorem 24)) *Consider the class of 1-d thresholds over the set $\mathcal{X} = \{1, 2, \dots, |\mathcal{X}|\}$. Consider any LU scheme for ERM for this class that works with arbitrary datasets of size at most n . Then, there exists a worst-case dataset such that the space complexity of the LU scheme is $\Omega(\min\{n, |\mathcal{Z}|\})$ bits. Furthermore, this lower bound holds even if we only consider unlearning queries of constant size.*

H.1. A Reduction from Unlearning Schemes for ERM to Realizability Testing

While Remark 51 shows that LU schemes for ERM only exist in very restricted settings, the implications for TiLU schemes are less clear.

In this section we discuss a white-box reduction to convert space complexity lower bounds for unlearning schemes for realizability testing to corresponding lower bounds for unlearning schemes for empirical risk minimization. At an intuitive level, it may seem that outputting the ERM of a dataset after having unlearning a subset of the points is a harder task than simply checking whether the dataset is $\mathcal{H}$ -realizable or not. While this intuition is certainly true if the input dataset D is $\mathcal{H}$ -realizable to begin with, and may also be true for many common hypothesis classes even ignoring this assumption, there are cases which this intuition breaks down. For instance, for the singleton hypothesis class $\mathcal{H} = \{h\}$, computing the ERM under unlearning queries can be done by an unlearning scheme with 0 space complexity; realizability testing on the other hand is a function of the labels of the datapoints and requires at least 1 bit of space. In spite of this pathological case, we show a way to reduce space complexity lower bounds on unlearning schemes for realizability testing to those for ERM that is applicable for a wide variety of hypothesis classes. We first begin by describing the general method we use to derive lower bounds for the former.

H.1.1. UNIFYING LOWER BOUNDS FOR UNLEARNING SCHEMES FOR REALIZABILITY TESTING

The lower bounds we prove in Theorem 7 and Theorem 10 follow a common recipe which we transcribe below. The lower bounds rely on constructing a collection of datasets $\mathcal{D} = \{D(z) : z \in \{0, 1\}^p\}$ indexed by p -bit binary strings where $D(z) = D(\emptyset) \cup \{(x_i, y_i) : i \in [p], z_i = 1\}$ for some base dataset $D(\emptyset)$ and $(x_i, y_i) \in \mathcal{X} \times \{0, 1\}$. The datasets in $\mathcal{D}$ satisfy the following structure: for each $i \in [p]$, there exists a subset of points $U_i \subseteq D(\emptyset)$, such that for any $z \in \{0, 1\}^d$, the dataset $D(z) \setminus (\{(x_j, y_j) : j \leq i - 1\} \cup U_i)$ is $\mathcal{H}$ -separable if and only if $z_i = 0$. The size of datasets in $\mathcal{D}$ are upper bounded by $|D(\emptyset)| + p$.

The lower bound on the space complexity of LU schemes for realizability testing follow by arguing that by following the unlearning queries $\{(x_j, y_j) : j \leq i - 1\} \cup U_i$ for $i = 1, 2, \dots, p$, the learner can recover the bits $z_1, \dots, z_p$ sequentially. For $n = |D(\emptyset)| + p$, this results in the lower bound,

$$s(n) \geq p, \quad (10)$$

for LU schemes for realizability testing. Likewise, for TiLU schemes for realizability testing, we arrive at the lower bound,

$$s(n) \geq \frac{p}{|\cup_i U_i|}, \quad (11)$$

We now discuss how to lift such lower bound instances for the case of unlearning for empirical risk minimization. We will make the reduction explicit for LU schemes; the same arguments also result in lower bounds for TiLU schemes for ERM.

H.1.2. THE WHITE-BOX REDUCTION

Recall the structural assumption we make on the lower bound instances for unlearning for realizability testing: for each $i \in [p]$, there exists a fixed unlearning query $U_i \subseteq D(\emptyset)$, such that for any $z \in \{0, 1\}^d$, the dataset $(D(z) \setminus \{(x_j, y_j) : j \leq i - 1\}) \setminus U_i$ is $\mathcal{H}$ -separable if and only if $z_i = 0$. From the collection of datasets $\mathcal{D}$ construct,

$$\mathcal{D}' = \left\{ D(z) \cup \{L \times (x_i, y_i) : i \in [p], z_i = 1\} : z \in \{0, 1\}^p, z_1 = 0 \right\} \quad (12)$$

where the notation $L \times (x, y)$ indicates L copies of the point (x, y) , and $L = \max_{i \geq 2} |U_i \setminus U_1|$. The datasets in $\mathcal{D}'$ have size at most $p(L + 2) + |D(\emptyset)| \leq (L + 3)n \leq 3n^2$ and are essentially indexed by a $p - 1$ -bit string $(z_2, \dots, z_p)$ since $z_1 = 0$.

Ignoring $i = 1$, we will start with $i = 2$ and assume that $z_2 = 1$. Since $z_1 = 0$, by the structural assumption on $\mathcal{D}$, the dataset $D'(z) \setminus U_1$ is realizable, and let $h \in \mathcal{H}$ be the realizing hypothesis. Therefore, on the dataset $D'(z) \setminus U_2$, the same hypothesis h incurs an error of at most $|U_2 \setminus U_1| \leq L$. On the other hand, note that any classifier on this dataset which classifies the point (x_2, y_2) incorrectly incurs an error of at least $L + 1$. These two conditions imply that when $z_2 = 1$, the ERM of the dataset $D'(z) \setminus U_2$ correctly classifies x_2 with the label y_2 . On the other hand, note that if $z_2 = 0$ then the ERM of $D'(z) \setminus U_2$ must label the point x_2 with the label $\neg y_2$. If this weren't the case, the dataset would remain $\mathcal{H}$ -realizable even upon adding (x_2, y_2) to it, which contradicts the structural condition on $\mathcal{D}$. In conclusion, the ERM of the dataset $D'(z) \setminus U_2$ evaluated on the point x_2 reveals whether the same dataset is $\mathcal{H}$ -realizable or not. If the predicted label on x_2 is y_2 , $D'(z) \setminus U_2$ is realizable, and if the predicted label is $\neg y_2$, the dataset is not realizable.

By recursing this argument, suppose $z_2, z_3, \dots, z_i$ have been inferred using the unlearning queries made thus far. Then the learner can compute z_{i+1} using a similar approach: the ERM on the dataset $(D'(z) \setminus \{(L + 1) \times (x_j, y_j) : j \leq i - 1, z_i = 1\}) \setminus U_i$ must correctly classify the point (x_j, y_j) if $z_i = 1$; on the other hand, if $z_i = 0$, the ERM on the same dataset must classify the point x_j as $\neg y_j$ if $z_i = 0$. The prediction of the ERM on the point x_j predicts z_{i+1} . By this construction, we see that an LU scheme for ERM is thus able to infer the hidden string $(z_2, z_3, \dots, z_p)$, which means that we must have $s_{\text{ERM}}(3n^2) \geq p - 1$. In comparison with (10), the space complexity lower bound is now on datasets of size $O(n^2)$. While these arguments are stated for LU schemes, extension to the ticketed case follow suite similarly.

As a result of this equivalence, we have a few corollaries in order.

Corollary 52 *For any hypothesis class with eluder dimension ϵ the space complexity of any LU scheme for ERM on datasets of size at most n satisfies,*

$$s_{\text{ERM}}(n) = \Omega(\epsilon \log(n/\epsilon)) \quad (13)$$

Note the white-box reduction we described in Section H.1.2 does not preserve the bound on size of the unlearning queries; notably, in the realizability testing lower bound (corresponding to the datasets $\mathcal{D}$), we consider unlearning queries of the form $U_1, U_2 \cup \{(x_1, y_1)\}, U_3 \cup \{(x_1, y_1), (x_2, y_2)\}$ and so on, while in the ERM lower bounds (Corresponding to the datasets $\mathcal{D}'$), the unlearning

queries are of the form $U_1, U_2 \cup \{(L+1) \times (x_1, y_1)\}, U_3 \cup \{(L+1) \times (x_1, y_1), (L+1) \times (x_2, y_2)\}$ and so on, which are roughly $L+1$ times larger. However, note that in certain special cases, such as the case of d -dimensional linear classifiers, the lower bounds for realizability testing only consider unlearning queries of the form $U_1, U_2, U_3, \dots$, without having to delete copies of the inferred datapoints, $\{(x_j, y_j) : j \leq i\}$ (cf. the proof of [Theorem 34](#)). In these special cases, the white-box reduction establishing space complexity lower bounds for LU/ TiLU schemes for ERM also consider the same kind of unlearning queries, namely, $U_1, U_2, U_3, \dots$. In these special cases, the reduction preserves the size of the unlearning queries. For TiLU schemes under ERM for halfspaces, we have the space complexity lower bound

$$s_{\text{ERM}}((L+3)n) \geq \frac{p-1}{|\cup_i U_i|} \quad (14)$$

In the lower bound instances we consider for halfspaces, the unlearning queries U_i correspond to subsets of k points drawn from $\{(e_i, 1) : i \in [d]\}$. In particular, the hidden vector is of dimension $p = \binom{d}{k}$, while $L = \max_{i \geq 2} |U_i \setminus U_1| \leq \max k$, and $|\cup_i U_i| = d$. Together this results in a lower bound for the ticketed setting for empirical risk minimization which closely mirrors that established for realizability testing in [Theorem 36](#).

Corollary 53 *Consider the hypothesis class corresponding to d -dimensional halfspaces on the domain $\mathcal{X}$ defined in [\(6\)](#) of size $d + \binom{d}{k}$. Consider any TiLU scheme for ERM which answers unlearning queries of size at most k . Then, there exists a worst-case dataset of size at most $n = (k+3)|\mathcal{X}|$ such that,*

$$s_{\text{ERM}}(n) = \Omega\left(\frac{1}{d} \cdot \binom{d}{k}\right). \quad (15)$$